

What It Takes to Trust a Measurement: Instrument Failures in an LLM Malware-Analysis Pipeline

Abstract

Multi-agent LLM pipelines are being built to map malware evidence to MITRE ATT&CK techniques, and their components are rarely measured against the deterministic tooling they sit on top of. We evaluate one such pipeline, and report why several of our own measurements of it were wrong.

Scored per sample on the same binaries and the same ground truth, the full pipeline reaches F1 0.1160 where the sandbox it is built on reaches 0.1130 — paired +0.0030, 95% cluster CI $[-0.0353, +0.0344]$, at a resolution of 0.085. Negotiated multi-agent consensus does not beat a single judge at equal token budget ($-0.0161, [-0.1247, +0.0819]$, at $3.2\times$ the tokens). Verbal confidence, the number every deterministic gate consumes, ranks correct claims above incorrect ones at AUC 0.550 $[0.475, 0.671]$ — indistinguishable from chance.

Seven defects in our own instrument produced plausible results rather than errors, and a suite of 2,712 passing tests caught none of them. Four crossed a boundary with another server; three did not, and those three share a different shape — a measurement’s cause assigned rather than measured. The largest instance is this paper’s own statistics: every interval we published resampled rows when the observations were clustered, and correcting it widened our anchor interval 2.32-fold and turned two equivalence claims into bounds.

We contribute the measured negatives with the minimum detectable effect beside each, seven failure mechanisms with what found them, and a no-LLM baseline (F1 0.1526, $n=97$ over 24 families) without which none of these numbers refers to anything.

1 Introduction

Three claims organise this paper.

First: the architecture we built did not survive its own evaluation. The system is a multi-agent pipeline over static, dynamic and network evidence, with a corroboration cascade, two retrieval tiers, and deterministic confidence gating. Each component was motivated by a plausible mechanism. Measured at equal token budget against simpler alternatives, the multi-agent design does not beat a single judge; the corroboration set never changes a verdict; the confidence number the gates consume discriminates at AUC 0.550; and all three retrieval components are near-inert end to end despite working in isolation. We report these as results rather than as tuning opportunities.

Second: those nulls are only interpretable because we measured whether each mechanism fired. A deterministic confidence cap that fires on 0.82% of techniques produces an ablation whose null result says nothing about the cap. A priority hint that fires on 56.7% of samples produces one that does. We adopted **firing rate before effect** as a rule after the first case, and it changed which experiments were worth running.

Third, and the reason for the paper’s title: the instrument was repeatedly wrong in ways that looked like data. Over one week we found four defects at the boundaries between the pipeline and the servers it depends on. In each case a server answered successfully and answered about something else: an argument the agent never supplied arrived as an explicit null; a program reported as loaded — by name — never became the one being analysed; a refused load returned HTTP 200 with an error in the body; and two bounds designed to protect the analyst composed into a path that returned nothing at all. None was caught by 2,712 passing tests, because each needs a *second* case in one server lifetime, and a unit test writes one.

Every one of those four was caught the same way: **a number repeated where variation was expected.** A hint of exactly 2,575 characters for two unrelated binaries; call graphs identical to the character for samples of 241 KB

and 139 KB; 66 consecutive samples at 75,426 characters. The check costs one line, needs no ground truth, and we now report it beside every batch result.

Then three more, and none of them was at a boundary. An ablation varied a deterministic post-processing step and we attributed its output to a language model. A rank correlation over model size ranked a reasoning-configuration flag instead and recovered the published scaling coefficient almost exactly — agreeing with the literature, which is the direction a result is least likely to be questioned. And a documented output ceiling on the verdict model was renamed by our client library during serialisation to a key the inference server does not read, so a component we describe as bounded had never once been bounded. The detector above does not find these: their outputs *do* vary. Two were caught by a number too clean to believe, one by a study that recorded which failure branch each call took. Each had passing unit tests over the exact function concerned, because the defect was never in a function — it was in which measurement was attributed to which cause, or in which representation of a value crossed a boundary.

We report them because the correction is part of the result. Two published findings of ours were withdrawn or rewritten on their account, and a third was one report away from being published as a scaling law.

The cost of not having had it is concrete: **a completed 210-sample study is withdrawn from this paper.** It met the precondition for one of the defects, and its per-sample outputs were not retained, so whether it was affected cannot be established. We report the withdrawal rather than the result.

We do not claim the category, and we counter-searched the newest mechanisms before claiming them either. Silent failures in production LLM agent runtimes are already described and taxonomised [20]; “distinct inputs should produce distinct outputs” is an ordinary metamorphic relation [22]; that an inference-time constraint can invert model rankings and confound a comparison is established, with a 28.4-point reversal, in [24]; and the token-limit incompatibility between OpenAI-compatible servers is documented in their own issue trackers. Every mechanism we report sits inside a published class, and one of them — the configuration difference masquerading as a scaling result — is a domain instance of [24] rather than a new phenomenon.

What we add is the setting and the price: an **evaluation pipeline for security research**, where the artefact is not a degraded user session but a measurement that is wrong and looks right.

Contributions. Each maps to one section, and each is a measurement or a mechanism rather than a proposal.

1. **A no-LLM baseline for LLM-based ATT&CK mapping**, and the full pipeline scored against it on the same samples and the same ground truth — the comparison without which every F1 in this literature, including our own earlier ones, refers to nothing (§5.1).
2. **Four measured architectural negatives** — multi-agent consensus at equal budget, a corroboration cascade, two-tier attribution, and a confidence gate — each reported with the minimum detectable effect of the design that produced it, because a null without its resolution is unreadable (§5.2–§5.6).
3. **Seven instrument failures that produced plausible results rather than errors**, with the layer each occurred at and what actually found it. Four crossed a boundary with another server; three were inside our own code and share a different shape (§6).
4. **A one-line detector** — how many distinct outputs did N inputs produce — that found all four boundary failures, needs no ground truth, and is reported beside every batch result rather than written as a test (§6.7).
5. **A selection rule that follows from a provider accepting a parameter and not acting on it**: arms must be matched on their *measured* configuration, not their requested one (§6.5).
6. **The price, demonstrated rather than hypothesised**: one completed 210-sample study withdrawn because its question can no longer be asked of it, and four arms of a later ablation unattributable for the same reason one level up (§7.4).

2 Background and Related Work

2.1 Mapping evidence to ATT&CK

Automated ATT&CK technique extraction has moved from supervised classifiers — TRAM, rcATT, TTPDrill — through neural extractors such as EXTRACTOR, AttackKG and LADDER, to retrieval-augmented generation. **TechniqueRAG** [7] pairs off-the-shelf retrievers with an instruction-tuned re-ranker and fine-tunes only the generator, motivated by the field’s data scarcity: ATT&CK defines over 550 (sub-)techniques while roughly 10,000 annotated examples exist publicly. Its hierarchical successor injects the tactic→technique taxonomy as an inductive bias, filtering at tactic level first to cut the candidate space by 77.5% for a 3.8% F1 gain and a 62.4% latency reduction. **TTPrint** takes an orthogonal route, decomposing reports into atomic behaviours and retaining only candidates supported by both localised evidence spans and the MITRE definition.

Büchel et al.’s SoK [5] is the field’s own assessment. Re-evaluating 40+ systems in a unified setting, they report a performance ceiling existing approaches have not crossed, that existing solutions are “largely incomparable” because they use custom ontologies and inaccessible datasets, and — counterintuitively — that **traditional NLP approaches outperform modern embedder-based and generative approaches in realistic settings**.

Position. Our describe-then-map design removes technique-ID recall from the model entirely: the analyst describes behaviour and a deterministic index assigns the identifier. This is a domain instantiation of **Infer-Retrieve-Rank** [6], which established the general program — multi-step LM/retriever interaction that decouples inference from assignment over a many-thousand-class label space — reaching state of the art on three benchmarks with tens of labelled examples. Ours is stricter than [6] and than [7]: in both of those the language model still ranks or selects among retrieved candidates, whereas here it never emits an identifier and any correction is gated to the provably-safe invalid→valid case. **We do not claim the idea.**

What we contribute is a decomposition of the SoK’s insight. [5] reports that embedders lose; we measure *where*. Ranking quality and gate quality behave as separable axes here: dense embeddings rank better while their scores barely separate a correct pick from a wrong one, and a lexical index ranks worse while gating cleanly. Composing the per-axis winners beats either pure backend on both, measured on TRAM2 (N=4,913) and replicated on the independently annotated AnnoCTR corpus [15].

The separation itself is not ours, and an earlier draft of this section claimed it was. Abdallah et al. [16] evaluate retrieval **confidence** — AUROC for predicting query success — as a dimension explicitly distinct from retrieval effectiveness, find calibration “consistently weak across model families”, and conclude raw scores are “unreliable for downstream routing without additional calibration”. That is the general statement, over more retrievers than we test. What remains ours is narrower and specific: the **inversion** — that in this task the *lexical* backend is the better gate while the *semantic* one is the better ranker, so the two axes have different winners and can be composed. We found that inversion neither asserted nor excluded elsewhere, which is a weaker absence than the one we first claimed, and we say so.

We also report a result about correction policy. Auto-correcting *valid but weakly-aligned* identifiers damages 38% of already-correct labels to recover 21% of wrong ones. **The asymmetry is not a discovery:** grammatical error correction has studied over-correction for years and evaluates with F0.5 precisely because a false correction costs more than a missed one [17], and recent work targets the failure directly. Our contribution is the *mechanism* in this setting: correct-but-weak and wrong-but-valid identifiers are **not separable by an alignment score**, so the valid→valid swap cannot be safely tuned at any error rate — which is why the shipped policy is restricted to the invalid→valid case, where zero regression holds by construction rather than by tuning.

Finally, the failure that motivates removing identifier recall in the first place. Asked for an ATT&CK identifier it does not know, the model enters a degenerate loop, proposing and re-proposing wrong identifiers until the budget is gone. **The loop is not novel and neither is the inadequacy of the obvious remedy:** neural text degeneration is a studied phenomenon with competing established accounts — Welleck et al. locate the cause in the *likelihood objective itself*, which “assigns too much probability to sequences containing repeats”, while a later data-side analysis finds a strong correlation with **repetition already present in the training data** and argues that the earlier self-reinforcement explanations reduce to it [18]. The limits of a uniform repetition penalty are known either way — it damages tokens that legitimately recur. We reproduce that here and can add one engine-level detail, that the server honours `repeat_` penalty while silently ignoring three sibling parameters, which converts a tight loop into a slower enumeration

without fixing it.

What we can contribute is the **consequence**. The degeneration literature measures repetition rate and text quality; here the budget is exhausted inside an operational deadline, the verdict stage times out, and the analyst receives an **empty STIX bundle** — degeneration as a failure to *deliver*, not a failure of prose. We measure that channel directly (§1.7.1: empty bundles 6/17 versus 1/17). Constrained decoding would also solve the original loop, and we say so; the point is that the cheaper mitigation a practitioner reaches for first does not.

2.2 LLM agents over binaries

Agentic systems now drive reverse-engineering backends through tool interfaces, and the model tier splits sharply by role. Fine-tuned binary specialists are overwhelmingly small and open-weight — ReCopilot on Qwen2.5-Coder-7B, LLM4Decompile at 1.3–33B, Nova — while the *tool-driving* agents use frontier models. **TTPDetect** [8] is the closest system to ours: it identifies TTPs in **stripped malware binaries** using dense plus neural retrieval to narrow analysis entry points, then a function-level agent with incremental context retrieval, reporting 93.25% precision and 93.81% recall at function level and 87.37% precision on real-world malware.

Position. We do not claim binary evidence as an input modality; [8] holds that ground. Two differences remain and only one is currently defensible. First, [8] appears to demonstrate on a cloud model while our pipeline runs on a ~3B-active local MoE — a deployment claim, addressed in §4 rather than here. Second, [8] reasons at function level over decompiled code while we fuse six deterministic detectors, three domain analysts and a corroboration cascade; that is a different system, but “different” is not a contribution until the difference is measured.

The one gap we do occupy is tool-catalogue scale. Degradation of tool selection as the catalogue grows is documented generally — LongFuncEval reports a 7–85% performance drop as the tool count increases, and reducing an edge model’s catalogue from 46 to 19 tools cuts execution time by up to 70% — but has not been measured for a reverse-engineering catalogue, where schemas are large and tools semantically overlap. Our finding that exposing ~201 Ghidra MCP tools (~22k tokens of schema) is infeasible at this model scale, motivating a curated 20-tool allowlist with high-value tools invoked deterministically from code, is the domain instance. It is a single feasibility point rather than a degradation curve, and we say so.

2.3 Multi-agent consensus

Multi-agent debate has become a standard pattern, including in security: PhishDebate assigns four specialised agents to distinct views of a webpage under a moderator and judge, reporting 98.2% recall and improvements over single-agent and chain-of-thought baselines.

Two 2026 results change how such claims must be read. Tran and Kiela [11] hold the reasoning token budget constant and find single agents consistently match or beat multi-agent systems on multi-hop reasoning, supported by an information-theoretic argument from the Data Processing Inequality: under a fixed budget with good context utilisation, decomposition introduces communication bottlenecks that lose information. Bertalaníč and Fortuna [10] reach the same conclusion on 7–8B open-weight models, where debate consumed 2.1–3.4× more tokens for equal or lower accuracy, and name three failure modes — **sycophantic conformity** (modal adoption up to 85.5%), contextual fragility, and consensus collapse discarding correct answers already present in the generation pool.

Position. We do not claim that multi-agent analysis beats a single agent, and we report the equal-budget comparison rather than assuming it. Both [10] and [11] scope their negative results to *homogeneous* agents decomposing a *single* context, and both name the exception — degraded or noisy context — which is the regime a 600-import binary plus a sandbox report plus network capture occupies. Whether heterogeneous evidence-channel decomposition survives the control that homogeneous debate fails is the question our ablation asks.

Our contribution here is a failure mode adjacent to [10]’s. Sycophantic conformity arises from majority pressure among agents holding the same information. We observed a distinct mechanism: an analyst whose **evidence channel was empty** paraphrased its peers, and because the echo returned tagged with that analyst’s own domain, the corroboration counter read it as independent confirmation — promoting a single-source finding to *corroborated*

and weighting it above the source it was copied from. In a security pipeline the harm is not a wrong answer but a **fabricated independent confirmation**.

Finally, weighting evidence by source reliability and discounting non-independent sources is Dempster–Shafer theory, decades old. Our cascade is an instance of it with hand-chosen constants, and we treat that as a limitation: measuring five perturbations of those constants — including inverting the most- and least-trusted layers — changed the top-10 ranking on 10.6–27.5% of samples and the corroborated set on **none**, because the corroboration decision is a function of layer count alone.

2.4 Local deployment

Confidentiality-driven local deployment is established rather than novel. **REx86** [12] fine-tunes eight open-weight models for x86 reverse engineering explicitly because “cloud-hosted, closed-weight models pose privacy and security risks and cannot be used in closed-network facilities”, and validates with a 43-participant user study in which line-level comprehension improved significantly ($p=0.031$) while the correct-solve rate rose from 31% to 53% without reaching significance ($p=0.189$). The capability gap is quantified: the UK AI Security Institute places open-weight cyber capability 4–7 months behind the closed frontier, narrowing from 6–10 months, at roughly 45× lower cost per task.

Position. We inherit this constraint and cite it; we do not claim it. Our deployment contribution is narrower and practitioner-facing: on a hybrid-offload MoE, KV cache costs ≈ 10.85 KiB/token and **context length barely moves system RAM** because the cost is the offloaded weights — a closed-form estimate over-predicted KV by $\sim 4\times$ and was overturned by measurement — and speculative decoding gave no throughput gain on this architecture. We found no comparable deployment-economics reporting for a complete local security pipeline.

2.5 Evaluation practice

This work’s methodological results belong to an established critique tradition rather than preceding it. Arp et al.’s *Dos and Don’ts* identified ten pitfalls in ML-for-security; **Chasing Shadows** [9] is its direct successor, surveying **all 72** peer-reviewed LLM-security papers from 2023–2024 and finding that **every one** contains at least one of nine pitfalls, with only 15.7% explicitly discussed. Broader benchmark audits reach the same place from the construct-validity direction across 445 LLM benchmarks.

Position. Our instrument-validity finding — that a single-run parsed-claim count produced contradictory rankings of the same arms across decoding budgets — is a **worked example** inside [9]’s category, not a new observation, and we present it as such: the nuisance parameter identified, the inversion measured. Likewise our narrative result. That template-based generation favours faithfulness while neural generation favours fluency is textbook data-to-text; what differs is the mechanism. In our paired comparison both arms were perfectly faithful — zero hallucinated techniques, precision 1.0 — so the entire F1 deficit came from **coverage**, not from hallucination. The LLM did not invent; it omitted.

Two evaluation results appear to be unoccupied. First, a case-prior retriever that is genuinely good in its own vocabulary (F1 0.620 against a 0.424 label-frequency prior) but scores 0.111 against that prior’s 0.123 when queried the way production queries it, because query and corpus share only their boilerplate — dense retrieval being the standard remedy for vocabulary mismatch makes this a failure of the remedy, visible only in the retriever’s **score distribution** while top-k accuracy stayed healthy. A published negative RAG result in malware analysis exists [13], by a different mechanism — retrieved context diluting a signal-extraction task — and the requirement to compare against a trivial label-only baseline is long established [14]. Second, an advisory prompt hint whose measurable effect was **completion under an operational time budget** rather than mapping accuracy, reducing empty fallback bundles from 6/17 to 1/17; we found no work measuring generation completion as a channel through which prompt changes act.

Finally, temporal stability — and here the axis matters more than the result. There is a substantial literature on **temporal generalisation** in language models, and its finding is degradation: performance falls as evaluation moves

away from the training period, and scale does not close the gap [19]. That work varies the **distance between training time and test time**. Our study varies something else. The model is **fixed**, and what changes is **the era of the input binary** — which is concept drift in the malware-detection sense, applied to an LLM analyst, rather than temporal misalignment of the model itself.

On that axis we find no LLM-based equivalent, and the axis is worth naming even though **we no longer have a result to report on it**. Our $n=210$ drift study is **withdrawn**: it drove the full pipeline per sample against a long-lived shared Ghidra server that was never restarted, which is the precondition for the stale-program defect described in §6, and its per-sample outputs were not retained, so whether it was affected cannot be determined after the fact. The earlier earliest-to-latest figure is not restated here. What survives is the framing — input-era drift is a distinct axis from temporal misalignment — and a re-run under the corrected instrument is required before any bound is claimed.

2.6 Silent failure at tool boundaries

The paper’s methodological spine belongs to an area that already exists, and locating it there precisely matters more than claiming the ground.

The genus is described. *When Errors Become Narratives* [20] studies a production LLM agent runtime over eight weeks, documents **22 incidents with root-cause postmortems**, and defines the meta-pattern as “*a failure whose error signal never reaches a human in actionable form*”, observed at least 28 times. Its five-class taxonomy — environment and platform quirks, design-assumption mismatches, error swallowing and dilution, chained hallucination, operational omission — covers our three defects without strain: an unset optional argument sent as `null` and a `load_program` that does not change what the server is looking at are **design-assumption mismatches**; a refused load answering HTTP 200 is **error swallowing**. Adjacent work reaches the same place from the interface side: an empirical study of automatically wrapping REST APIs as MCP servers finds baseline generation succeeds for only **76% of sampled tools**, rising to 94.2% with automated repair [21] — an interface layer that is unfaithful to its source often enough to be a measured quantity rather than an anecdote.

The detector is not new either. “Distinct inputs should produce distinct outputs” is an ordinary metamorphic relation, and metamorphic testing exists precisely for programs whose correct output is unknown [22] — the “notorious oracle problem” its own survey names. Stress-testing an LLM judge’s consistency under input variation is likewise established: [23] perturbs formatting, paraphrase, verbosity and ground-truth labels and measures how the judgment moves.

And the newest of our mechanisms is a domain instance, not a discovery. In the course of building a parameter-size series we produced a rank correlation of +0.866 between model size and F1 — reproducing the $\rho=0.85$ the nearest prior work reports — from five rows that were three models, one of them present twice at two reasoning settings. The low-scoring row was a model disabled by a flag rather than limited by its size, and it sat at the small end of the axis where it set the sign. That phenomenon is established: *Brevity Constraints Reverse Performance Hierarchies in Language Models* [24] shows an inference-time output-length constraint **inverting** a 28.4-point gap between large and small models, and frames it exactly as a methodological problem — universal evaluation protocols mask latent capability, so protocols must adapt to the model. A reasoning flag consumes the answer budget, which makes our disabled arm a brevity-constrained arm under another name, and a reversal is a stronger result than a spurious correlation. We report ours as an instance of theirs.

What does not appear in [24], and is what we would add, sits in the remedy rather than the phenomenon. Their protocol is adapted per model; ours has to be **verified per arm**, because one of our providers accepts the parameter that would match two arms and does not act on it — so an arm can be labelled matched while running the opposite configuration. Selection has to key on the *measured* reasoning share of a completed arm, never on the flag that was requested. The same is true one layer down: that OpenAI-compatible servers disagree about which token-limit parameter they honour is documented in their own issue trackers [25], so what we contribute there is not the incompatibility but what one costs inside a measurement — a documented output ceiling absent for as long as it had existed, and half of a study’s calls diverted onto a different artefact-construction path.

What we add is therefore narrow and stated narrowly. Three instances of the described genus, drawn not from

a user-facing assistant but from an **evaluation pipeline for security research**, where the consequence is neither a bad answer to a user nor a degraded session but **a measurement that is wrong and looks right**. We contribute: the three concrete mechanisms at three different integration boundaries (MCP argument encoding, server-side current-program state, HTTP status versus body); the output-cardinality check reported *alongside* the result rather than run as a test; and a demonstrated cost — one study withdrawn, because its per-sample outputs were not retained and the question can no longer be asked of it. The last of these is the part we have not seen elsewhere: the taxonomies report incidents, and we report what an incident of this class does to a result that has already been written down.

One further claim, and it is about where the genus ends rather than what is in it. [20]’s five classes each describe a call to something else — an environment quirk, a swallowed error, a mismatched design assumption at an interface. Three of our seven mechanisms have no interface in them at all: an ablation that varied a deterministic post-processing step and attributed the result to a language model; the correlation described above; and an output ceiling correct at every level of our own object model and wrong only in the serialised request. No server misled us in any of the three. They share the meta-pattern [20] names — the error signal never reaches a human in actionable form — while sitting outside every class it offers, because the taxonomy is organised by the boundary crossed and these crossed none. We suggest the organising principle that covers both is not the boundary but the attribution: **the same shape appears wherever a measurement’s cause is assigned rather than measured**. We state that as a proposal, on seven cases from one project, not as a taxonomy.

2.7 Scope of every empirical statement above

Each of our results cited in this section was produced on **one model on one machine**: Qwen3.6-35B-A3B at IQ3_K_R4 quantisation, served by ik_llama.cpp llama-server on a single RTX 5060 / 31 GiB host, with the exact model revision, GGUF digest and engine commit recorded in the reproducibility appendix. Where a statement concerns retrieval or the deterministic cascade rather than generation, no model was involved at all and the binding limit is the index and the corpus instead; those are marked at the point of claim.

We state this here rather than only in Threats to Validity because [9]’s **surrogate fallacy** is the pitfall this work is most exposed to, and because the nearest external evidence sharpens it: in an ATT&CK-classification study across models, **parameter size was the only statistically significant predictor of F1** ($\rho=0.85$, $p=0.014$) while prompt strategy, chain-of-thought and temperature were not. Single-model findings are therefore exactly the kind that should not be read as properties of an architecture. Where a result is a *negative* obtained under a configuration favourable to the alternative, it travels further than a positive; we say which is which at each claim.

2.8 Reference note

[5]–[14] are as numbered in findings-log.md §References. [15] is the AnnoCTR corpus (Lange et al., LREC-COLING 2024, arXiv:2404.07765, CC-BY-SA 4.0), added here for the §1 replication. Systems named without a bracketed number — TRAM, rcATT, TTPDrill, EXTRACTOR, AttacKG, LADDER, PhishDebate, LongFuncEval, ReCopilot, LLM4Decompile, Nova — are cited via the verified sources that discuss them and **must be resolved to their own records before submission**; see the citation audit for which were fetched directly and which were not.

[16]–[19] were added on 2026-08-09 by the A4 adjacent-field counter-search, and their verification status differs — **deliberately recorded rather than smoothed over**:

Table 1: Sources added by the adjacent-field counter-search, with how each was verified. Three were cited for something the source does not say.

#	Source	Verified how
[16]	Abdallah, Holdcroft, Ali & Jatowt, <i>Are LLM-Based Retrievers Worth Their Cost?</i> , arXiv:2604.03676	Abstract fetched and read. The quoted claims are from it
[17]	The F0.5 convention in grammatical error correction: Ng et al., <i>The CoNLL-2014 Shared Task on GEC</i> ; Bryant et al., <i>The BEA-2019 Shared Task on GEC</i>	Re-anchored and verified 2026-08-15. The claim is that GEC weights precision over recall because a false correction costs more than a missed one, and the shared tasks are its primary source: F0.5 is the main metric in both, weighting precision twice recall, chosen to penalise over-correction. The two candidate papers previously listed here were never fetched and are dropped — the convention is documented by the tasks themselves and does not need them
[18]	Welleck, Kulikov, Roller, Dinan, Cho & Weston, <i>Neural Text Generation with Unlikelihood Training</i> , arXiv:1908.04319; Li, Lan, Fu, Cai, Liu, Collier, Watanabe & Su, <i>Repetition In Repetition Out</i> , arXiv:2310.10226	Both abstracts fetched 2026-08-15 — and our summary of them was wrong. We had written “an established account rooted in self-reinforcement and training-data repetition”. Welleck’s account is neither: it blames <i>the likelihood objective itself</i> . The data-side account is the second paper’s, which finds a strong correlation with repetition in the training data and argues self-reinforcement explanations reduce to it. Corrected in the body to two named accounts rather than one blended one. The third id previously listed (arXiv:2505.10402) was never fetched and is dropped

#	Source	Verified how
[19]	Lazaridou, Kuncoro, Gribovskaya, Agrawal, Liska, Terzi, Gimenez, de Masson d’Autume, Kocisky, Ruder, Yogatama, Cao, Young & Blunsom, <i>Mind the Gap: Assessing Temporal Generalization in Neural Language Models</i> , NeurIPS 2021, arXiv:2102.01951	Abstract fetched and read 2026-08-15; supports the claim exactly. Both halves are quoted from it: “ <i>model performance becomes increasingly worse with time</i> ” and “ <i>while increasing model size alone — a key driver behind recent progress — does not solve this problem</i> ”. The arXiv id is recorded now because it was missing and a first attempt to verify this row fetched the wrong paper. TemporalWiki and TARDIS were never fetched and are dropped

[20] | Wei Wu, *When Errors Become Narratives: A Longitudinal Taxonomy of Silent Failures in a Production LLM Agent Runtime*, arXiv:2606.14589 | **Abstract page fetched and read.** Title, sole author, the 22-incident/8-week scope, the “*error signal never reaches a human in actionable form*” definition and the five-class taxonomy are all from the paper. The full text has **not** been read, and the abstract does not itself claim coverage of HTTP-status-versus-body or stale-state failures — our mapping of M1–M3 onto its classes is **our reading and must be checked against the body before submission** |

[21] | *From REST to MCP: An Empirical Study of API Wrapping and Automated Server Generation for LLM Agents*, arXiv:2507.16044 | **Abstract fetched and read, 2026-08-15 — and it corrected us.** The sentence we had quoted, “*silent downstream reasoning failures rather than explicit, testable errors*”, **is not in the paper**; it came from a search summary, exactly as the previous row warned. Removed. What the abstract does state is quoted now instead: 76% of sampled tools wrap successfully, 94.2% after automated repair. The two companion arXiv ids previously listed here were never fetched and are dropped rather than carried |

[22] | Li, Liu, Poon, Towey, Sun, Zheng, Zhou & Chen, *Metamorphic Relation Generation: State of the Art and Research Directions*, ACM TOSEM 10.1145/3708521 (preprint arXiv:2406.05397) | **Verified 2026-08-15.** Title, authors, venue and DOI confirmed; the abstract states metamorphic testing addresses “*the notorious oracle problem*”, which is what we cite it for. The ACM page is paywalled to the fetcher, so the preprint is given alongside for a reader |

[23] | Dev, Sloan, Kavner, Kong & Sandler, *Judge Reliability Harness: Stress Testing the Reliability of LLM Judges*, arXiv:2603.05399 | **Abstract fetched and read, 2026-08-15 — and it does not say what we cited it for.** We claimed it established “duplicating dataset items to verify identical inputs score consistently” as eval-harness practice. It does not: it perturbs formatting, paraphrase, verbosity and the ground-truth label, and measures how the judgment moves. That is adjacent, not the same, and the claim has been rewritten to what the source supports. **We have no verified source for the duplicate-item practice and therefore no longer assert one** — which weakens a demotion of our own detector, i.e. moves in the direction that flatters us, so it is flagged here rather than quietly enjoyed |

[24]–[25] were added on 2026-08-15 by the counter-search for M6 and M7. Both demoted a claim of ours before it was made, which is the point of running the search first:

Table 2: Sources added by the counter-search for the last two mechanisms. Both demoted a claim of ours before it was made.

#	Source	Verified how
[24]	<i>Brevity Constraints Reverse Performance Hierarchies in Language Models</i> , arXiv:2604.00025	Abstract fetched and read. The 28.4-point underperformance of larger models under spontaneous verbosity, the 7.7–15.9-point inversion under brevity constraints, and the framing of universal evaluation protocols as masking latent capability are all quoted from it. The full text has not been read; our claim that our reasoning-flag arm is a brevity-constrained arm under another name is our reading and must be checked against the body before submission
[25]	Token-limit incompatibility across OpenAI-compatible servers: ggml-org/llama.cpp issue #8634 (max_tokens not respected on the non-chat endpoint, generation continuing to the context limit); vllm-project/vllm issue #11976 (request for a server-side cap)	Search results only. The issues have not been read in full. They are cited to <i>demote</i> our M7 from a new mechanism to a known integration wart, which is the safe direction; a submitted paper needs them read

All of [17]–[25] have now been fetched (2026-08-15), and three of the nine were wrong. [21] was quoted for a sentence not in it; [23] was cited for a practice it does not describe; [18]’s mechanism was a blend of two incompatible accounts. Each is corrected above, and the citations that were never fetched at all — five arXiv ids carried as company for the ones that were — are dropped rather than kept as decoration.

Acting on [17]–[23] now is conservative because each one *demotes* a claim of ours — the safe direction. Citing them in a submitted paper on this basis would not be, and the standing rule from the 2026-08-08 citation audit applies: a search summary is a lead, not a source. That audit exists because one fabricated a claim.

And one nearly did again, 2026-08-11. While counter-searching §6, a search summary asserted that a paper documented harness-level silent failures misattributed to the model and resolved by auditing platform source code — close and useful prior art, had it existed. **Both candidate papers were fetched and neither contained it.** The summary was a plausible synthesis of several results. The rule held only because it was applied; it is recorded here because the failure it prevents is the one this section is about, one level up.

3 The System, Briefly

3.1 Shape

The pipeline maps evidence about a malware sample to MITRE ATT&CK technique IDs and emits a STIX bundle. Three analysts run over heterogeneous evidence channels, a judge synthesises a verdict, and a deterministic layer adjudicates between them.

Table 3: The pipeline’s components and what each contributes, as built. Whether each contributes anything measurable is the subject of Section refsec:results.

component	what it does	verdict
static analyst	ReAct loop over a headless reverse-engineering server (decompilation, imports, strings, call graph), 20 tools from a curated allowlist	operational; its salvage path was defective (§6)
dynamic analyst	ReAct loop over a CAPEv2 sandbox via MCP	operational; only Windows PE is analysable on our instance
network analyst	local PCAP analysis over the sandbox’s capture	operational; depends on the sandbox indirectly
judge	synthesises a verdict and emits STIX	operational
corroboration cascade	weights layers, marks a technique corroborated when ≥ 2 layers contribute	measured: never reaches the artefact (0/32); a reconciliation step restores every cascade technique the judge omits, and across 80 arms the judge added none of its own
confidence gating	caps confidence for unsupported obfuscation/injection claims	measured: fires on 0.82% of techniques
case-prior RAG	retrieves similar past cases to prime mapping	measured: loses to a label-frequency prior in production
family-feature RAG	retrieves family fingerprints	measured: +0.0029 F1 end to end
opcode-hash attribution	normalised function hashes matched against a corpus	measured: fires on 0 of 18 samples
sink-reachability hint	ranks functions reachable to sensitive APIs, injected as prompt steering	fires on 56.7% ; measured: $\Delta\text{techniqueIDs}$ +0.50, CI [−3.33, +4.50]

Two design choices are worth stating because they are *not* what failed, and a reader should not infer that everything did.

Tool exposure is not tool availability. The reverse-engineering server advertises ~165 tools. Exposing the full catalogue to a 3B-active-parameter MoE is infeasible — the model’s tool selection degrades before the context does — so the analyst is given a curated 20. This is a deliberate narrowing with a measurement behind it, and it held.

The pipeline degrades rather than fails when a service is absent. With the sandbox unreachable the dynamic path is skipped and the run completes on static evidence, which is pinned by a test. This mattered during the study: the sandbox was reachable only from one network, and every static-only result in this paper was produced by that degradation working as designed.

3.2 What runs it

One local model — Qwen3.6-35B-A3B at IQ3_K_R4, served by ik_llama.cpp on a single RTX 5060 host with hybrid MoE offload — plus a headless reverse-engineering container, a Qdrant vector store, and a CAPEv2 sandbox on a separate machine. Exact identifiers are in the reproducibility appendix, and their limits are not incidental: the model server’s memory grows with cumulative requests, generation rate varies eightfold with context length on this architecture, and the sandbox retains its reports for days. Each of those shaped an experiment reported here.

3.3 Why the system is not the contribution

Of the four claims the architecture was built around — multi-agent consensus over heterogeneous channels, a multi-layer corroboration cascade, two-tier attribution, and falsification-before-confidence — **all four are now measured, and every one is negative or near-null**. The last of them no longer awaits sandbox access: the cascade was closed on the recovered cohort, and the attribution tier’s remaining half was measured at 0 of 18 samples. The components are not broken; several work well in isolation and were built for sound reasons. They simply do not move the end-to-end result, and we could not have known that without measuring each mechanism’s firing rate before its effect.

That outcome is the reason this paper is about measurement. A system whose parts individually justify themselves and collectively change nothing is an ordinary result, and reporting it requires only honesty. What required method was discovering that several of those measurements had been wrong in the first place.

4 Measurement Methodology

4.1 Equal budgets, with a noise control

Arms are compared at an **equal total token budget**. A multi-agent arm that makes N calls receives B/N per call against a single arm’s one call of B , so a win cannot come from spending more. The design follows Bertalaníč & Fortuna, including a **stochastic-noise control** in which one analyst receives evidence irrelevant to the sample.

The control is not decoration: it establishes that the harness can detect a difference. In our consensus study the noise arm separated clearly (0.3366 against 0.3975 and 0.4136) while the treatment did not, which distinguishes “no effect” from “no sensitivity”.

For reasoning models the budget must cover reasoning. Our frontier endpoint reports reasoning tokens separately from content, and **56.5%** of its output was reasoning across 25 real prompts (84% on a one-token answer). Capping content alone would have granted it roughly twice the generation for the same nominal budget.

And the arms must be on the same side of the reasoning switch, which is a stronger requirement than it sounds. We measured this flag on two models at 25 calls each: enabling it moved F1 by **0.45** on one and **0.34** on the other, in both cases because the model spent its entire output budget thinking and returned no answer — 24 of 25 calls hit the cap on the second model, and one answered. That is larger than every architectural effect in this paper combined, so an unmatched pair does not produce a noisy comparison; it produces a comparison of the flag. Two further cautions follow from what it took to hold this constant. The flag cannot be read from the request: one of our providers accepts the parameter and ignores it, returning the same 56% reasoning share with and without it, so **matching must be verified against the measured reasoning share of the completed arm**. And it cannot be assumed uniform across a series: when we ranked arms by parameter count without checking, the ranking recovered the published scaling correlation ($\rho=+0.866$) from an axis on which one point was a disabled model rather than a small one.

4.2 Paired designs, bootstrap intervals, and never a single run

Every arm sees the same samples in the same order; differences are computed **per sample and then aggregated**, with 95% bootstrap confidence intervals over the paired deltas.

The motivating failure is specific. An early study ranked prompt structures using a **single-run parsed-claim count**, and the ranking **inverted** when the decoding budget changed — the instrument was measuring the interaction of structure and budget while being read as measuring structure. A claim count from one run is not an instrument, and we now say so where that result is cited.

4.3 Firing rate before effect

Before ablating a mechanism, measure **how often it engages**. The rule came from two cases that look identical in a results table and are not:

Table 4: Firing rates measured before any ablation was run, and what each rate licenses the ablation to say.

mechanism	fires on	what an ablation would mean
confidence cap	0.82% of techniques	nothing — a null describes the 99.2% where it never ran
sink-reachability hint	56.7% of samples	a real statement about the hint

The cap’s own preconditions explain its rate — three technique families, only when the sole contributing layer is static, and 84% of those claims are YARA-only, so no static claim exists for it to discipline. Reporting the rate is more informative than an ablation of it, and cheaper.

A corollary: an ablation must be **restricted to the samples where the mechanism fires**, and must report the remainder separately. Averaging a feature over inputs it never touched dilutes a real effect toward zero and manufactures a null.

4.4 Output cardinality, reported beside every batch

How many distinct outputs did the N inputs produce? If far fewer than N differ on a dimension that should vary, the instrument is repeating itself. The four *boundary* defects in §6 were found this way and none by a 2,712-test suite. The three that are ours rather than a server’s were not: their outputs do vary, and what found them was a number too clean to believe, an arms table read under a correlation, and a study that recorded which failure branch each call took.

It is reported as a result column, not run as a test, because the signal exists only in the batch: each individual response was well-formed and plausible. We report 50 distinct call-graph sizes across 79 samples, 63 across 97.

4.5 Per-sample outputs are retained for every study

Aggregates cannot be re-interrogated. When a defect was found that *might* have affected a completed 210-sample study, the question could not be asked, because only the summary survived — and the study is withdrawn as a direct consequence.

The policy is therefore not “keep the numbers” but **keep enough to re-ask a question that has not been thought of yet**: per-sample predictions, the resolved ground truth, timings, and the identifier of every external task the run depended on.

4.6 Fresh server state per sample

Shared, long-lived servers accumulate state that crosses sample boundaries. In our sweeps, restarting the reverse-engineering server between samples **recovered 12 of 14 samples previously recorded as unmeasurable** — the failures belonged to the state each sample inherited from its predecessor, not to the samples themselves. A read timeout leaves the JVM mid-analysis and every subsequent load is refused; without restarts, one slow sample silently invalidates the window that follows it.

The same applies to the model server: it grows with cumulative requests and does not plateau (10.4 GB fresh, 14.8 GB after one pass), so long paired runs restart it between arms. At temperature 0 this is measurement-neutral.

4.7 Two audits that cost minutes and caught real errors

Diff the claim register against the evidence log. Both documents are maintained; that is exactly what allows them to diverge. Applied twice here, it found a claim recorded as novel one hour after our own text called it a replication, an audit row mis-attributing a model caveat to a server-free harness, and three ledger rows still carrying superseded figures.

Counter-search each novelty claim in an adjacent field’s vocabulary. A claim of novelty is a *searched absence*, and searching in one’s own field’s words finds nothing by construction. Five claims were demoted this way — including the one central to this paper, when the silent-failure genus turned out to be already taxonomised. Two rules learned from doing it:

- A search summary is a lead, not a source. One asserted prior art that neither candidate paper contained when fetched; citing it would have put a fabricated reference in the ledger.
- The result of a counter-search is recorded whether or not it demotes anything, because “we looked and found nothing” is only credible if the looking is documented.

4.8 What this costs

These practices are cheap individually and expensive together, and the expense is honest to report. The firing-rate rule adds a measurement before every ablation. Per-sample retention adds storage (~660 MB of sandbox reports for one cohort). Fresh state per sample adds ~20 seconds of restart to every sample. Output cardinality adds one line.

Against that: one withdrawn study, seven defects that produced plausible wrong data, and three retrieval components that would have been reported as working had they only been tested in isolation. On this project the practices paid for themselves several times over, which is the only argument for them we can make from a single system.

5 Results

5.1 The baseline, first

Every accuracy number in this work is read against one reference, and we state it before any of our own. **CAPE alone — the sandbox the pipeline is built on, mapping its signature hits to ATT&CK technique IDs deterministically, with no language model anywhere:**

Table 5: The no-LLM baseline. CAPE’s own signature-derived technique identifiers, scored against family-level MITRE ground truth. Intervals resample **families**, not samples: ground truth is resolved per family, so two binaries of one family are scored against a byte-identical label vector and are not two observations.

	mean	95% cluster CI	ICC	effective n
precision	0.2413	[+0.1850, +0.3010]	0.584	35.0
recall	0.1328	[+0.0896, +0.1810]	0.638	33.0
F1	0.1526	[+0.1114, +0.1998]	0.692	31.2

n = 97 samples over 24 families, mean 4.04 samples per family; bootstrap seed recorded with the interval. CAPE asserted at least one technique on **every** sample, so this is a predictor rather than an artefact of an empty one. Ground truth resolution uses the same alias map as the drift harness, so the two studies score identically.

We report these intervals at the family level because our first version of this table did not, and the difference is not decorative: resampling samples rather than families gave an F1 interval 2.32 times narrower than the one above, on data whose intra-cluster correlation is 0.692. The row count is 97; the count that supports an inference is 31.2.

And here is what the pipeline adds to it. On the 13 samples that completed both arms of the paired study of §5 — the only samples where pipeline and baseline have been run on the same inputs — all three scored per sample against the same ground truth by the same code:

Table 6: Pipeline against the engine it is built on, like for like.

	mean F1	95% cluster CI
CAPE alone, no language model	0.1130	[+0.0654, +0.1624]
pipeline, static evidence only	0.1130	[+0.0770, +0.1543]
pipeline, with the sandbox report	0.1160	[+0.0823, +0.1485]

n = 13 samples over 8 families. Paired, because every sample went through both: **+0.0030 F1**, 95% cluster CI [−0.0353, +0.0344], exact cluster sign-flip p = 0.9609, better on 7 of 13.

Three analyst agents, a negotiation loop, a revision pass, an LLM judge and a weighted corroboration cascade land within +0.0030 F1 of the signature engine they are built on top of. Two of the means agreeing to four decimal places is a coincidence, and was checked rather than reported: the samples differ individually in both directions. The system is not reproducing the sandbox’s answers — it reaches different answers of the same quality.

The interval on that difference is what should be read, not the point: at 8 families this design could detect a difference of **0.085 F1** at 80% power, and it did not detect one. That is a bound on the pipeline’s contribution, not a demonstration that the contribution is zero.

Why the cohort is 97 and not 100, and why the reason is not the one we first wrote down. The cohort was submitted as one batch and the sandbox reported every task as complete. It had not been. Querying each task’s timing shows a split with nothing between the modes: the 43 tasks whose reports survived ran for 186–366 s, and the other 56 ran for **under a second**, all 56 of them still marked reported, none with a report directory. A Windows PE does not detonate in one second. The ordering implicates the instrument rather than the samples — real analyses run through one afternoon and every task after that returns instantly — and re-submitting the same binaries from the same local files two days later produced full-length analyses on the same instance. Nearly all were recovered that way; three failed in processing or reporting and are permanently gone.

We had originally written that the analyses ran and their reports had expired. That was the comfortable reading of Reports directory does not exist, and it was wrong. The correction matters beyond the sample count: a completion status from this instrument is not evidence that anything happened, which is why the retrieval path now checks each analysis’s wall-clock duration before accepting its report (§E6).

We report the baseline first because without it a pipeline F1 is uninterpretable, and because it sets a bar the pipeline must clear to have contributed anything. Earlier drafts of this work reported F1 values against nothing at all.

The two panels do not share an axis, and that is the point. An earlier version of this figure put all five arms on one scale, where a no-LLM baseline near 0.1526 beneath treatment arms near 0.4136 read as a large pipeline win. It is not one: the left panel’s arms run on 5 synthesised inputs whose evidence is generated from their own technique lists and are scored against per-sample truth, while the baseline runs on real binaries scored against family-level truth. Different inputs, different truth granularity, different attainable ceilings. No baseline is definable for the left panel at all — a deterministic regular expression over the dictionary that generated its evidence scores 1.000 there by construction.

The right panel is the comparison that can be made: the 13 samples over 8 families that completed both arms of §5, with CAPE scored on the same binaries against the same ground truth by the same code. The three intervals overlap, and the paired pipeline-minus-baseline difference is +0.0030 [−0.0353, +0.0344] on a design able to resolve 0.085.

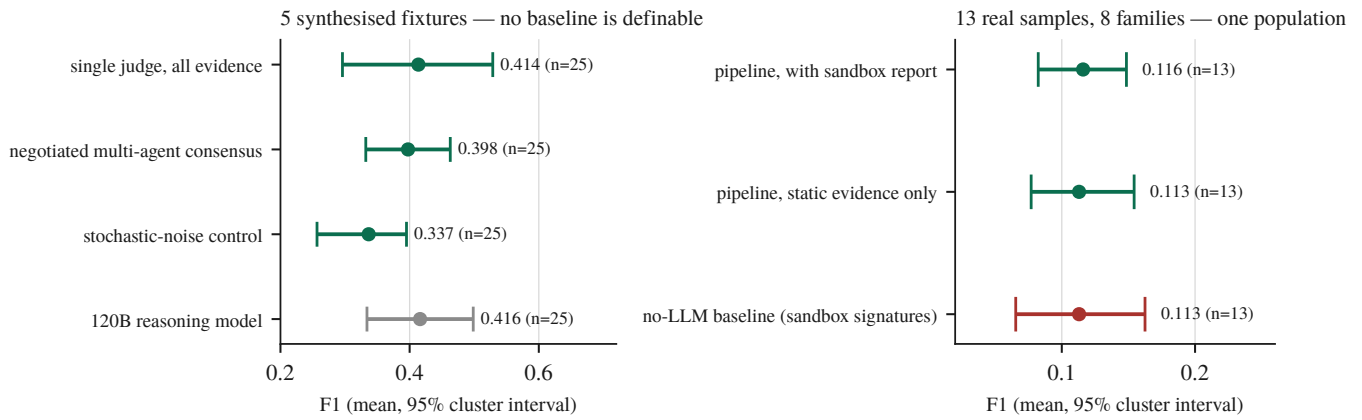


Figure 1: Figure 1: On the one population where pipeline and baseline are comparable, they overlap. Points are means over per-sample F1; bars are 95% intervals resampling the cluster the observations are independent at, recomputed from the retained per-sample records with the seed fixed.

5.2 Multi-agent consensus does not pay for itself at equal budget

Design after Bertalaníč & Fortuna: three arms at an **equal total token budget**, including a stochastic-noise control in which one analyst receives irrelevant evidence.

Table 7: The equal-budget consensus arms. n is rows; the unit that supports an inference is the sample, of which there are 5.

arm	mean F1	rows	samples
single judge, all evidence at once	0.4136	25	5
negotiated multi-agent consensus	0.3975	25	5
noise control	0.3366	25	5

Paired delta, negotiated – single: -0.0161 , 95% cluster CI $[-0.1247, +0.0819]$ — an interval containing zero, at **3.2× the tokens**. At 80% power this design could have detected a difference of **0.223 F1**; the observed difference is two orders of magnitude smaller than that. The null is a bound on what the design could see, not a demonstration that the two arms are equivalent.

What the noise control does and does not license. Earlier drafts read the control’s separation as proof that the harness can detect a difference when one exists. It does not carry that weight. The control moves by -0.0770 , $[-0.1337, -0.0233]$ — every one of the 5 samples in the same direction — but the effect is *below this design’s own minimum detectable effect* of 0.120, and the exact cluster permutation test returns $p = 0.0625$. That is the smallest p the design can produce: with 5 clusters the two-sided sign-flip test floors at 0.0625, so **no comparison on this corpus can reach $\alpha = 0.05$ whatever its effect size**. The honest statement is that all 5 samples moved the same way, which is worth reporting and is not a demonstration of sensitivity.

This was pre-registered as the experiment that would decide the paper’s framing, and its null result is why this is not a systems paper. The literature’s prior ran the same way: two 2026 equal-budget studies find single agents match or beat debate, and arXiv:2604.02460 §5.3 reports a crossover at $\alpha=0.7$ on our own model class.

5.3 A model 3.4 times larger does not separate from the local one

Same five fixtures, same prompt, same 2,400-token output budget:

Table 8: Local against frontier, same fixtures, same nominal budget.

arm	model	mean F1	rows	samples
local	Qwen3.6-35B-A3B (IQ3_K_R4)	0.4136	25	5
frontier	Nemotron-3-Super- 120B-A12B	0.4162	25	5

Both arms see the same fixtures at the same repeats, so the comparison is **paired**:

frontier – local = +0.0026, 95% cluster CI **[–0.1322, +0.1460]**, 25 rows over 5 samples. The frontier model is better on 12 and worse on 13.

A 3.4× larger reasoning model, given the same evidence and the same nominal output budget, lands within three thousandths of F1 of a 35B model quantised to run on one desktop GPU, and its direction across samples is a coin flip.

That null is not a claim of equivalence, and the interval is the reason. At 80% power this design could have detected a difference of **0.300 F1** — coarser than every architectural effect reported in this paper, and coarser than the largest configuration effect we found. A design that could only ever have seen effects above 0.300 says nothing about the +0.0026 it returned. We report the null because it is what we measured and because it is the direction the literature’s prior already pointed; we do not report it as evidence that the two models are alike.

We have since found that the two arms were not configured alike, and we report the null with that caveat attached. The local arm runs with its reasoning stream disabled — necessarily, because the local server otherwise returns an empty answer — while the frontier arm ran with reasoning on and spent 56.5% of its budget there. The nominal budget was equal; the budget available for an *answer* was not. How much this matters we can bound from a third endpoint measured later on the same fixtures: with reasoning enabled it spent 99.9% of every call’s budget thinking and scored **0.0000** across all 25 calls, and with reasoning disabled the same model scored **0.4501**. That flag is worth **+0.4501 F1**, [+0.3604, +0.5437] — the largest effect measured on any arm in this paper. The frontier arm above was not crippled that way (it parsed 25 of 25), so the null is not an artefact of a collapsed arm; but it is not the single-variable comparison it appears to be.

The matched arm cannot be built on that endpoint, and we established this by measurement rather than by assumption. We re-ran the 120B arm with the reasoning parameter set. The provider accepted it and ignored it: 56.2% of the output was still reasoning, against 56.5% without it, and the arm returned 0.4149 where the first run returned 0.4162 (paired $\Delta = -0.0014$, 95% cluster CI [–0.0695, +0.0799]). The re-run is therefore a replication of the original configuration, not a correction of it. The confound in this comparison is a property of the endpoint, and we report the null with the confound stated rather than promising a cleaner version of it.

A matched comparison is available on different weights, and it bounds two things at once. The model we run locally, qwen3.6-35b-a3b, is also hosted by its vendor at full precision on an endpoint that does honour the flag. Against our local 3-bit deployment, on the same fixtures and repeats:

Table 9: The same weights at two precisions, with and without the reasoning flag. Paired against the local arm; intervals resample samples.

arm	precision	reasoning	mean F1	paired vs local	95% cluster CI
local	IQ3_K_R4 (3-bit)	off	0.4136	—	—
vendor-hosted	full	off	0.3507	–0.0629	[–0.2133, +0.0963]

arm	precision	reasoning	mean F1	paired vs local	95% cluster CI
vendor-hosted	full	on	0.0080	−0.4056	[−0.5232, −0.2880]

Two results follow, and the first is weaker than we first wrote it. **We cannot detect a quantisation penalty, and we cannot rule out a large one:** the paired difference is −0.0629 with an interval of [−0.2133, +0.0963], which admits a penalty of over 0.2 F1 against our local deployment, and this design’s minimum detectable effect is 0.331. What the data support is that 3-bit quantisation is not *demonstrably* this pipeline’s handicap, not that it is not one. Second, **the reasoning flag replicates at +0.3427 paired** ([+0.2008, +0.4936]) on a second model — this time the one we host — with 24 of 25 calls exhausting the output budget on reasoning. The flag is the only effect in this paper that exceeds its own design’s minimum detectable effect (0.310 here, 0.200 on the third endpoint). The largest effect we have measured on any arm is a configuration flag, not an architecture and not a parameter count.

Two qualifications belong with that, because it is the paper’s one positive result. The flag comparisons are **post-hoc** — they were run after the confound was discovered, not planned — and are corrected as a family of 5, at which their q-values are 0.1042 and 0.1042. And like every comparison on this corpus they sit at the exact test’s floor of 0.0625. They are reported as sign-consistent across all 5 samples with an effect several times their detection threshold, which is the strongest statement this design can support and is not a claim of significance.

We report the frontier null because the literature’s prior predicts otherwise: arXiv:2606.18166 found parameter size the *only* statistically significant predictor of ATT&CK-classification F1 ($\rho=0.85$, $p=0.014$) on the nearest task. On this task, at this budget, we do not reproduce that — and we cannot test it as a series either: a rank correlation over model size needs the arms matched on the flag that is worth more than the size effect, and the only endpoint we have above 35B does not permit that match. Two of our arms are configuration-matched and both are 35B, so the series is two points at one size. We state this as a limitation with a measured cause rather than as an untested claim in either direction.

An earlier version of this arm reported 0.5025 and was wrong to suggest a lead. That figure came from $n=9$, the run having been cut short by a daily request quota; the apparent advantage did not survive completing the sample. It is recorded here because it is the exact failure mode §4.2 warns about — a difference read off an underpowered arm — and because we caught it by finishing the run rather than by any insight.

The completed run parsed **25 of 25** calls with no refusals and one hitting the output cap.

Where the larger model’s budget went: across the 25 calls, **56.5% of output tokens were reasoning**, and on a one-token answer, 84% — 92 output tokens, 77 of them thinking. An equal-budget comparison must therefore cap *total* output including reasoning; capping content alone would hand the reasoning model roughly twice the generation for the same nominal budget, and the null above would have been a win bought with tokens.

5.4 Three independently built retrieval components, all near-inert in production

This is the paper’s most replicated result, and it is negative.

Table 10: Three retrieval components, each measured in isolation and again end to end. Every one works where it was built and moves nothing where it is wired in, and the three fail for three different reasons.

component	works in isolation?	contribution end to end
ATT&CK case-prior RAG	yes — F1 0.620 vs a 0.424 frequency prior	loses to the frequency prior in production (0.111 vs 0.123)
family-feature RAG	yes — recall@5 0.199, 6.3× chance , leakage-free split	+0.0029 F1 , precision −0.0088, $n=19$
opcode-hash attribution	n/a — never fires	0 of 18 samples ; 7,716 functions hashed, 0 matches

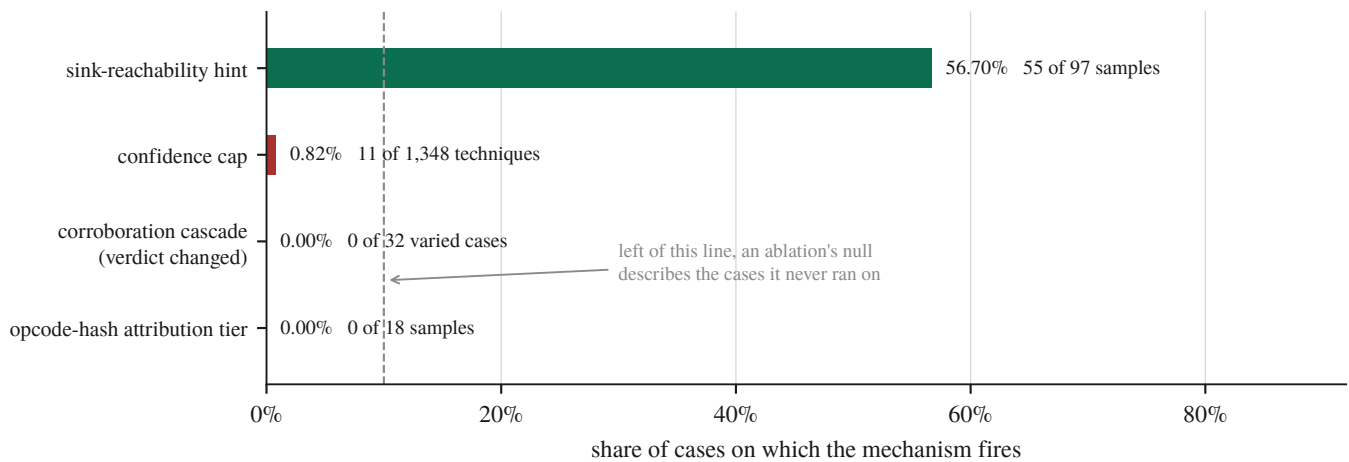


Figure 2: Figure 2: Firing rate decides whether an ablation can be read at all. Three of the four mechanisms the architecture was built around engage on almost nothing, and an ablation of any of them would return a null describing the cases where the mechanism never ran. Only the sink-reachability hint clears a rate at which an ablation would carry information — which is why it is the one we ablated, and §7 reports the null it returned.

Each was built separately, each is defensible in design, and each is inert once wired to real inputs. The mechanisms differ, which is what makes the pattern worth reporting rather than three disappointments: the first fails on **vocabulary mismatch** between query and corpus, visible only in the score distribution while top-k accuracy stayed healthy; the second is simply small; the third fails for **two independent reasons** — its corpus holds 3 samples under 2 generic labels, and the 8-instruction floor that correctly excludes thinks leaves **9 of 18 real samples with one hashable function or none**, a sharply bimodal distribution that populating the corpus would not fix.

5.5 Mechanisms measured before their effects

A recurring finding is that a mechanism’s *firing rate* determines whether its ablation can be interpreted at all.

Table 11: Firing rate before effect. A mechanism’s ablation can only be read where the mechanism runs, and two of these run too rarely for an ablation to say anything.

mechanism	fires on	consequence
confidence cap (falsification-before-confidence)	0.82% of techniques	an ablation would measure nothing; the null is uninterpretable
sink-reachability priority hint	56.7% of samples (55/97)	an ablation is interpretable, was run, and returned a null (§7)
STIX integrity pass	measured over 60 fresh judge bundles	reported by removal reason

For the cap, the mechanism’s own preconditions explain the rate: it applies to three technique families, only when the sole contributing layer is `static`, and 84% of those claims are YARA-only — so no static claim exists for it to discipline. We report the rate rather than an ablation, because an ablation over a mechanism that fires on 0.82% of cases produces a null that means nothing.

5.6 The corroboration cascade does not reach the verdict

The multi-layer corroboration set is the design’s central claim about evidence quality. We remove one Layer-0 source at a time and compare the bundle the analyst receives against the bundle produced with all of them, under two conditions that differ in exactly one respect — whether the removed source was the sole owner of its techniques.

Table 12: The corroboration cascade under two conditions. Both counts follow from the reconciliation step rather than from the verdict model, which is why they are arithmetic rather than evidence.

condition	what removing a source does to the evidence	verdict changed	Jaccard vs all-sources
disjoint	its techniques disappear; nothing else claims them	32/32	0.738–0.765
overlap	its techniques survive under a partner source, but their corroboration changes	0/32	1.000 [1.000, 1.000]

Take a technique away and the bundle loses it. Leave the technique in place and destroy the cross-domain agreement behind it — the thing the cascade exists to compute — and the bundle is byte-identical. The cascade runs, weights each source by a trust coefficient, and reports a corroborated set in the run summary; none of that reaches the artefact the analyst is given.

Why the bundle behaves that way is not what we first wrote, and the difference matters. Our initial reading was that the judge attends to the claim list and ignores everything else. It does not read either. A post-processing step reconciles the bundle against the cascade: every attack-pattern whose technique ID cannot be resolved is dropped, and every cascade technique missing from the bundle is appended. The cascade’s technique set is therefore a guaranteed subset of every bundle the system emits, independent of what the judge produced. Checked against all 80 stored arms by recomputing each arm’s cascade set from the same seeded fixtures: **the bundle is exactly the cascade’s technique set in 80 of 80, and the judge contributes zero techniques to any of them.**

Both rows of the table then follow from set arithmetic. Under overlap, removing a source whose claims a partner also makes leaves the cascade’s set unchanged, so the bundle cannot change — 0 of 32 is a necessity, not a measurement of restraint. Under disjoint, removing a source deletes its techniques from that set, so the bundle shrinks — 32 of 32, equally necessary. The Jaccard of 1.000 with a zero-width interval was the tell we missed: a language model at temperature 0, asked 32 times, does not usually agree with itself perfectly. We read a constant as a strong null.

The architectural conclusion is unchanged and, if anything, sharper. Corroboration does not reach the analyst’s artefact. But the reason is not that a model overlooked it: **the judge cannot subtract from the bundle’s technique set, and across 80 arms it added nothing to it.** Omissions are restored by reconciliation; additions would have survived, and there were none.

We stated that bound rather than the stronger claim it invites, because the evidence could not distinguish a judge whose output was unusable and wholly replaced from one that reproduced the cascade’s set exactly. **That measurement has since been taken**, by intercepting the reconciliation step and recording what the model produced before the deterministic set was merged in. On the four calls that reached it:

Table 13: What the verdict model contributed to the artefact the analyst receives, intercepted at the reconciliation seam. Counts are techniques, over four calls.

	total	per call
attack-patterns the judge emitted	50	12.5
carrying a resolvable ATT&CK ID	12	3.0
dropped — the model named no technique	38 (76.0%)	9.5
IDs the cascade did not already hold	0	0.0
injected because the judge omitted them	87	21.8

Three of the four calls produced nothing nameable at all; the fourth named twelve techniques, every one already in the cascade. Three quarters of the model’s own attack-patterns asserted a behaviour it could not map to a technique and were discarded. Of the two pipelines the bound allowed, the evidence points at the second: the bundle is the cascade’s set wearing the judge’s name.

And half the calls never reached that step. Four of the eight timed out at the verdict ceiling, and on a timeout `give_verdict` returns a text-fallback bundle that never calls the reconciliation routine — so the cascade is not consulted on that path at all. The timeouts are not a property of the fixtures: the judge’s 8,192-token output cap was renamed by our client library to a parameter the local inference server does not read, so the model decoded unbounded until the caller gave up (\$6, M7). One such call was measured at **30,155 generated tokens** and was still going.

Repeating the study with the cap fixed turns the pair into a controlled experiment, and the result inverts the section’s own conclusion. With the ceiling binding, the same four fixtures fail and the same four succeed; the judge’s share of the bundle is 0.0% in both conditions; every number on the reconciled path is identical. What changes is only how the failure arrives — 8,192 tokens of unparseable output in three and a half minutes, instead of a ten-minute timeout.

But the artefact is not the same. The fallback builder scrapes ATT&CK IDs from two places: the evidence claims, and the model’s own raw response. In the uncapped condition that response is the literal string [TIMEOUT] and contains no IDs, so the fallback bundle was exactly the evidence-derived set — identical to the cascade set on all four calls. In the capped condition it is 18,748 to 24,710 characters of degenerate decode, and:

Table 14: Techniques reaching the analyst on the text-fallback path against those the corroboration cascade held, per fixture. The last column is the set no evidence source corroborated.

fixture	fallback	cascade	only in fallback
jhuugit	32	20	12
sardonic	27	25	2
sliver	46	23	23
wannacry	26	16	10

47 techniques reach the analyst that the corroboration cascade never held, and none goes the other way. On one sample the bundle doubles. Because the uncapped run is a control whose raw text provably contains no IDs, every one of them is attributable to the model’s own output. They passed through no filter at all — not the cascade, not the reconciliation step, not the invalid-ID filter, not the integrity pass.

They are also, checked against the 691-technique ATT&CK catalogue, **real: 45 of 45 unique IDs exist, none is invented**. That is worth stating because it removes the easy reading. The pipeline is not emitting hallucinated identifiers a schema check would catch. It is emitting genuine ATT&CK techniques that no evidence source claimed and no corroboration supports, in the same object type and the same bundle as techniques three independent sources agreed on, with nothing downstream to tell them apart.

So the finding that opened this section needs its scope stated exactly. The verdict model has no influence over which techniques reach the analyst **on the path where it works**. On the path where it fails it has more influence than on the path where it succeeds: 0 of 99 techniques when reconciliation runs, 47 of 131 when it does not. The architecture suppresses the model’s contribution precisely when the model is functioning, and admits it precisely when it is not.

This replaces an earlier version of the same result, and the replacement is why we trust it. The first pass varied three of six Layer-0 sources — the three needing no sandbox — over fixtures carrying five techniques, and reported the verdict unchanged in 0 of 15 cases. Two defects were found in it afterwards. The absent sources included the Sigma layer, which fires on 94 of 97 samples at weight 0.55, above every static layer that study did vary; and at five techniques over six sources, round-robin assignment left one source with nothing, so its arm was identical to the baseline by arithmetic rather than by measurement. The re-run uses fixtures of 12 to 51 techniques so every source

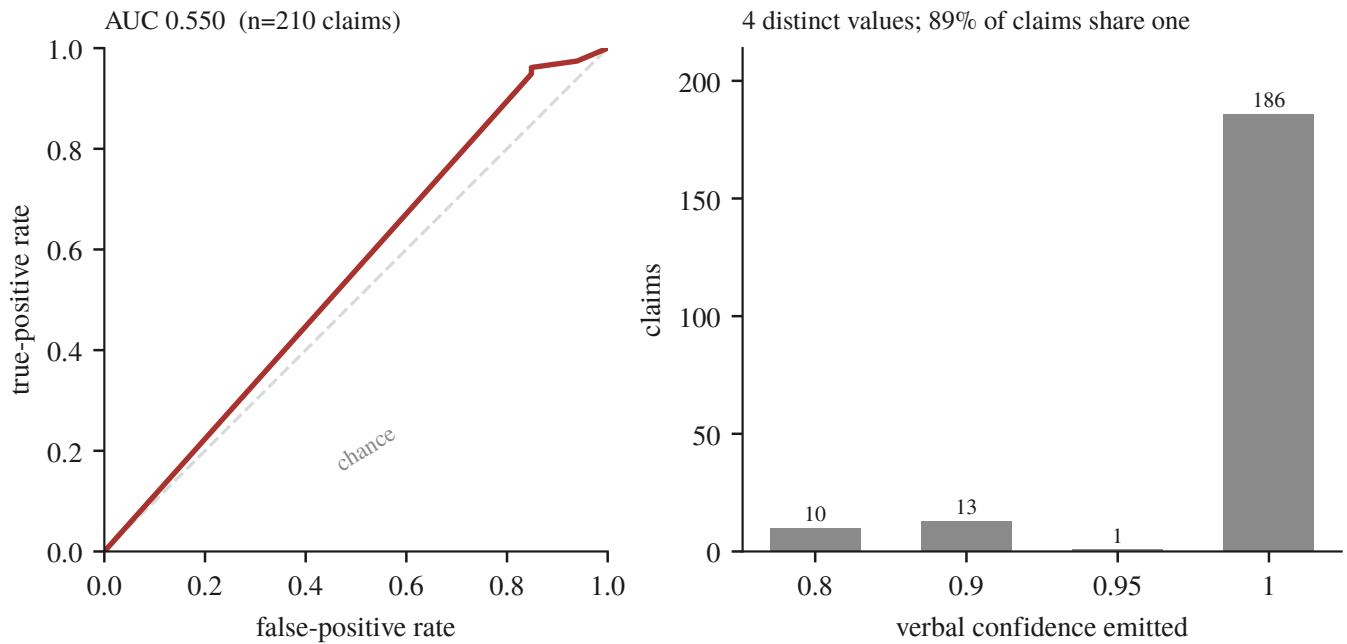


Figure 3: Figure 3: The number the gates consume barely orders anything. Left, the ROC over 210 claims scored against resolved ground truth. Right, why: **186 of those 210 claims carry confidence exactly 1.0**, so the score has almost no ordering to contribute. The estimate is rank-based with ties averaged, which for this distribution is not a detail — sorting the tied claims arbitrarily instead returns 0.458, and the difference between the two numbers is entirely an artefact of how 89% of the data is handled.

carries at least three claims, includes the Sigma layer, and adds a corroborated source pair that does not involve YARA so that no arm’s result is a foregone conclusion. The null survives all of it, at 32 arms rather than 15.

One pre-registered prediction, resolved in both directions. We predicted that removing the tool-artifact layer would change nothing, because it emits on YARA’s cascade domain and therefore cannot contribute corroboration. In the overlap condition that is exactly right: 0 of 8. In the disjoint condition it is wrong: 8 of 8, because there that source solely owns its techniques and removing it removes them. The prediction was about corroboration and it holds precisely where corroboration is the only thing varying — which is the sharpest evidence we have that the two conditions measure what we claim they measure.

Two Layer-0 sources are absent from these arms by measurement rather than by choice. Over the 95 archived reports the LOLBin layer and the DGA layer each produce a claim on **0 of 97**, while being fed a median of 8888 recorded API calls and 48–68 domains per sample respectively. They are offered the evidence and decline it; giving either an equal share of the ground truth would have ablated a mechanism that never engages in this deployment.

5.7 Confidence is very nearly a constant

Verbal confidence, which the cascade and the deterministic gates consume, discriminates correct from incorrect claims at **AUC 0.550** — near chance — and is concentrated on a handful of round values. This matches arXiv:2606.29490’s finding that verbal confidence measures willingness to commit rather than correctness, and it means every deterministic gate keyed to that number is keyed to noise.

5.8 What the ablations cost, and what that revealed

The one mechanism whose firing rate justified an ablation (§4, 56.7%) was ablated, paired, on the subset where it fires. It does nothing measurable:

Table 15: The sink-reachability hint, paired on the samples where it fires. Units differ per row and are named in the row.

outcome	mean Δ (hint on – off)	95% CI
distinct technique IDs	+0.50	[−3.33, +4.50]
claims	−0.83	[−4.33, +3.67]
seconds	+52.55	[−161.93, +268.45]

n=6 pairs; the hint is better on 2, worse on 2, and tied on 2, with per-pair deltas of +4, −6, 0, 0, −4 and +9 — large in both directions and cancelling. This is the fourth architectural claim to end in a null, and the last one we were in a position to test.

The result that constrains the result: half the experiment was not scoreable. Twelve pairs were attempted and six survived screening.

Table 16: Every sample excluded from the paired study, with the reason and the count. Reported because a loss rate bounds what any ablation on this pipeline can detect.

excluded	n	why
degenerate decode	3	an arm emitting 49–117 claims across 2–14 technique IDs (ratios 8.4–34.3)
unattributable	2	both arms dead, and the host state needed to decide whether the pipeline or the machine failed was never recorded
incomplete	1	an arm exceeded the 630 s bound — a genuine outcome, but it costs the pair

A 50% pair-loss rate bounds what any ablation on this pipeline can detect. An effect smaller than the noise from losing half the samples is not measurable here, and quoting +0.50 [−3.33, +4.50] without that context would claim a precision the instrument does not have. **Reporting the loss rate beside the estimate is the whole of what we have to say about how to read it.**

The two unattributable pairs are this paper’s own \$4.5 recurring against it. They died while the host’s swap file was exhausted and the model server had 2.3 GB of its own address space paged out; one arm then exceeded a 594-second budget on a prompt trimmed to 16,000 characters, which is inexplicable for a model generating from RAM and unsurprising for one generating from disk. We had retained per-sample outputs, as our own rule requires. What went unrecorded was the environment the measurement ran in. The harness now captures MemAvailable, SwapFree and the server’s resident-versus-swapped split at both ends of every arm and the scorer screens on it — forward only, never for data already collected.

Three arms failed outright; two were re-run and one deliberately was not. Two carried a Connection error from a restart race, meaning the measurement never happened, and were re-run. The third was the 630-second timeout, which is an outcome — deleting an outcome one dislikes is selection rather than repair. The recovered pair contributed a −4, against the hint.

Running it, however, surfaced a production defect that the test suite (2,712 passing) did not: on any binary rich enough to exhaust the 40-step ReAct budget, the analyst returned **zero techniques**, because the salvage path received a fresh copy of a budget it was already inside. Fixed and verified on the same sample: **1,677 s → 323 s, 0 → 5 technique IDs**. The fix is partial — one sample still exceeds the bound, and the server’s own timings show generation varying from 162 to 20 tokens/s on this hybrid recurrent architecture, so a bounded input does not rescue a case where the tokens themselves are eight times slower than assumed.

5.9 Summary

Of the four architectural claims this system was built around, **all four are now measured and every one is negative or near-null**. None is left pending sandbox access: the cascade closed on the recovered cohort and the attribution tier's remaining half fired on 0 of 18 samples. What survived measurement is not the architecture but the account of measuring it.

6 Instrument Failures

An LLM analysis pipeline is mostly other people's servers. Ours calls a reverse-engineering server for disassembly, a sandbox for detonation, a vector store for retrieval, and a model server for inference, over three protocols. Each boundary is a place where a request can succeed and still not mean what the caller assumed.

This section reports seven such failures. We report them together because they share a shape that a test suite is poorly positioned to catch and a results table cannot show: **the instrument answered successfully, and answered about something else**. The last three are ours rather than someone else's server — two in the analysis code that produces our results, one in the pipeline that produces the artefact — which is the point: the shape does not stop at the network boundary, and it does not stop at the evaluation harness either.

6.1 Seven mechanisms

6.1.1 M1 — An unset argument arrives as an assertion

The sandbox's MCP interface declares 36 tools. Every one takes an optional `token` parameter with a declared default of `""`. Our client built its argument model with “required, else None”, and the agent framework fills every declared field before invoking. So an argument the agent never mentioned reached the server as an **explicit null**, and a field typed `str` rejected it:

```
1 validation error for call[verify_auth]
token
  Input should be a valid string [type=string_type, input_value=None, ...]
```

All 36 tools failed. *Unset* and *null* are different statements, and the client was making the second while intending the first.

Why it survived to first contact. The reverse-engineering server — the only one this client had ever talked to — declares no optional parameter with a typed default, so the same malformed argument dictionary happened to be acceptable there. A defect in shared client code was invisible because only one of the two servers it served had ever been exercised.

6.1.2 M2 — A load that does not change what the server is looking at

`load_program` imports a binary and answers `{"success": true, "program": "<name>"}`. Every caller read that as “the server is now looking at this binary”. The server maintains a separate notion of the **current program**, and `load_program` sets it only when nothing is current yet:

```
load A      -> {"success": true, "program": "A"}    current: A
load B      -> {"success": true, "program": "B"}    current: A    <-- still A
run_analysis -> {"program": "A", "new_functions": 0}
call graph  -> A's graph, byte-identical, for every sample
```

From the **second sample of a container's lifetime onwards**, everything derived — decompilation, imports, strings, call graph, function hashes — described the first binary while the report named the current one. The response even contained the correct program name, which is what made it convincing.

6.1.3 M3 — A refusal wearing a success

The same server answers a load it cannot perform with **HTTP 200** and an error in the body:

```
{"error": "Failed to load program from: /data/samples/x.exe"}
```

`raise_for_status()` sees nothing wrong. Downstream code then analysed and described whichever program remained current. When the server began refusing loads after roughly thirty in one lifetime, **66 consecutive samples produced a call graph of exactly 75,426 characters** before anyone noticed.

6.1.4 M4 — Two bounds composing into an empty result

The analyst's ReAct loop has a 40-step budget; exceeding it triggers a salvage that re-invokes the model to synthesise from what was gathered. That salvage received a **fresh copy of the full time budget** — so loop-plus-salvage exceeded the hard cap by construction. On any binary rich enough to exhaust the step budget the analysis returned **zero techniques**, at a cost of 28 minutes:

```
loop completed: 19 tool calls, 41 messages, elapsed 109.5s  
ended without a final answer ... forcing synthesis  
static no-tools fallback exceeded the 1530s hard cap
```

Neither bound was wrong alone. Exceeding the first *guarantees* an attempt at the second, and the composition was never considered.

6.1.5 The same mechanism at a fifth boundary — caught before it produced data

M1–M4 were all found after the fact, in data already collected. The fifth was not, and the difference is the whole argument of this paper.

Fetching the sandbox cohort's reports, we found that a request for a report the sandbox has since deleted is answered with **HTTP 200** and a 63-byte body:

```
{"error":true,"error_value":"Reports directory does not exist"}
```

Fifty-six of one hundred tasks answer this way — the analyses ran, and their reports have aged out of retention. A fetcher that trusted the status code would have written fifty-six of those bodies into the archive under the filename of the sample they were supposed to describe, and the three studies that read that archive would each have scored them.

They did not, because the fetcher was written to verify the report's own `target.file.sha256` against the identifier it asked for before writing anything. That check exists only because M3 taught us the shape: an HTTP status is a statement about the transport, not about the answer. **This is the first time the discipline paid forward rather than backward**, and it cost four lines.

We first recorded the consequence as a retention limit: the reports had existed and expired. That explanation was wrong, and finding out how wrong is the fifth failure in this list. Asking the sandbox how long each analysis had taken produced a split with nothing between the modes — the 43 tasks whose reports survived ran 186–366 s, and the other 57 ran **zero to one second**, 56 of them still marked reported. A Windows PE does not detonate in one second. Nothing had expired; nothing had happened, and the instrument said otherwise.

Re-submitting the same binaries from the same local files two days later produced full-length analyses on the same instance, which recovers 54 of the 57 and puts the cohort at 97. Three failed in processing or reporting and are gone for good.

The lesson generalises past this instrument. A completion status is a statement about a queue, not about an analysis, and it is exactly as trustworthy as an HTTP 200. The retrieval path now reads each task's wall-clock duration

and refuses anything under 60 s — an order of magnitude below the shortest real analysis on this instance — before it will accept a report. We would rather report a smaller honest n than a full one built on error bodies, and we would rather state the reason we verified than the one that first sounded plausible.

6.1.6 M5 — An ablation that measured a code path and reported a language model

The four above are failures of other people’s servers. This one is ours, it is the most recent (2026-08-14), and it is the only one that reached a results table.

We ablated the corroboration cascade by removing one evidence source at a time and comparing the STIX bundle the analyst receives. Under the condition where a removed source’s techniques survive under a partner — so that only their *corroboration* changes — the bundle was identical in **32 of 32** arms, at Jaccard 1.000. Under the condition where the removed source solely owned its techniques, **32 of 32** changed. We wrote this up as a finding about the judge model: that it attends to the list of claimed techniques and ignores the evidence-quality apparatus above it.

The judge was not involved. A post-processing step reconciles the bundle against the cascade before it is returned: unresolvable attack-patterns are dropped, and **every cascade technique missing from the bundle is appended to it**. The cascade’s technique set is therefore a guaranteed subset of every bundle the pipeline emits, whatever the model produced. Recomputing each arm’s cascade set from the same seeded fixtures shows the bundle is *exactly* that set in **80 of 80** arms, with the model contributing **zero** techniques to any of them.

Both results then follow from set arithmetic. Removing a source with a duplicate partner does not change the cascade’s set, so the bundle cannot change; removing a sole owner does, so it must. The two numbers we reported are necessities of an if statement, and the experiment could not have returned anything else.

The architectural conclusion survives the correction — corroboration does not reach the analyst’s artefact — but the mechanism is not the one we described. The model cannot subtract from the bundle’s technique set and, across 80 arms, added nothing to it. Whether its output was unusable and wholly replaced, or happened to reproduce the cascade’s set, this evidence cannot say; both leave the same trace, and separating them needs the pre-reconciliation output measured directly. What is certain is narrower and sufficient: the model has no influence over which techniques reach the analyst. And a second study, already written and about to run, was designed to compare a weighted cascade against a flat union of the same claims. Both of its arms would have shared one reconciliation set, so it would have reported “no difference” after an hour of computation, correctly and vacuously. It was stopped four minutes in, by the same log line that exposed M5.

What made this one hard to see. The tell was in the results the whole time: a Jaccard of 1.000 with a zero-width bootstrap interval, from a language model asked the same question 32 times at temperature 0. Models do not agree with themselves that well. We read a constant as an unusually clean null, because a clean null was the result we were prepared to find.

6.1.7 M6 — A configuration difference wearing a parameter count

The most recent (2026-08-14), and the only one whose wrong answer **agreed with the literature**.

We assembled a parameter-size series to test a published finding that parameter count is the only significant predictor of ATT&CK-classification F1 ($\rho=0.85$). The harness read every completed arm file, ranked mean F1 against total parameters, and reported $\rho=+0.866$ — reproducing the prior almost exactly, across a 3× span, with the confounds it could not remove printed honestly beneath.

The five rows were three models. One model appeared twice because it had been run in two configurations, and another twice because it had been run twice. The duplicate configurations were not a labelling detail: the same weights score **0.3507** with the reasoning stream disabled and **0.0080** with it enabled, because 24 of 25 calls then spend their entire output budget reasoning and never answer. The 0.0080 row sat at the small end of the parameter axis, where it set the sign of the correlation. What the series measured was a flag, ordered by coincidence against size.

The failure is not the duplicate rows; it is that the arm-selection rule did not exist. The harness had careful

arithmetic — averaged ranks so file order cannot break ties, an exact permutation p because four points cannot reach significance, a common-cell restriction so endpoint availability cannot masquerade as model size — and every one of those guards was about a way the *numbers* could mislead. None was about whether the rows were comparable in the first place.

The rule now keys on the **measured** reasoning share of each arm rather than the flag the harness requested, because those two disagree: one provider accepts the parameter and ignores it, so an arm selected on intent would enter the series labelled matched while running the opposite configuration. With the rule applied, two arms qualify and both are 35B, so the series refuses and says which arm was excluded and at what measured reasoning share. Nine unit tests pin the rule.

What made this one hard to see. It agreed with the published result. M5 was caught by a number too clean to believe; this one produced a number in exactly the range a reader would expect, in the direction the literature predicts, with its limitations already stated. There was nothing anomalous to notice. It was caught by reading the arms table under the correlation and asking why one model was on it twice — which is a question no result, however wrong, would have prompted.

And the phenomenon is not ours. arXiv:2604.00025 shows that an inference-time output-length constraint *reverses* performance hierarchies between large and small models [24] — a 28.4-point gap inverted — and frames it as a methodological confound requiring evaluation protocols that adapt to the model rather than a universal one. Our reasoning flag consumes the answer budget, which makes the disabled arm a brevity-constrained arm under a different name, and a reversal is a stronger result than a spurious correlation. We report M6 as a domain instance of that finding.

What we would add is narrower and sits in the remedy rather than the phenomenon: their protocol is adapted per model, ours has to be **verified per arm**, because matching on the requested configuration is not sufficient when a provider accepts the parameter and does not act on it (§3.32). An arm can be labelled matched and be running the opposite setting. The rule that follows — select on the *measured* reasoning share, never on the flag — is what the gate implements.

6.1.8 M7 — A safety property that was configured, documented, and never sent

The last one is in the production pipeline rather than in an evaluation harness, and it removed a guarantee the code states in its own comment.

The verdict model is built with an 8,192-token output ceiling, and the line that builds it says why: “*Bound the verdict generation so a degenerate decode can’t consume the full wall-clock timeout.*” The client library renames that parameter to the API vendor’s newer spelling when it serialises the request, and our local inference server does not read the newer spelling. It accepted the field, ignored it, and decoded without a ceiling.

Measured on one call: a 1,403-token prompt, **30,155 tokens generated**, still generating at 46 tokens per second when the caller’s ten-minute wrapper gave up. The only thing that ever stopped a verdict was wall-clock.

The consequence is not a slow call. Four of eight fixtures in a separate study never returned a verdict at all, and for each of them the pipeline emitted a bundle through its text-fallback path — which does not run the reconciliation step, so the corroboration cascade contributed nothing and the analyst received techniques copied straight from the raw evidence claims. A component we describe as bounded was unbounded, and its failure mode was to hand the analyst a differently-built artefact without saying so.

Where it was invisible. The configured value was correct. The container passed it. The model object held it. It survived every later rebinding intact. Only the serialised request was wrong, and nothing in the system reads the serialised request. Every check anyone would think to perform would have confirmed the property that did not hold.

What found it. Not a test and not a review. A study measuring something else — what the verdict model contributes to the bundle — recorded *which branch* each failed call took rather than only that it failed. Four calls named the timeout branch, which prompted the question of why a temperature-zero model would need ten minutes, which led to the server’s own log and a token counter that had passed thirty thousand. The instrumentation that caught it was written for a different question three hours earlier.

The incompatibility itself is known; the consequence is what we contribute. That OpenAI-compatible servers disagree about token-limit parameters is documented in their own issue trackers — `llama.cpp` #8634 reports generation continuing to the context limit when the cap is not honoured on one of its endpoints. We are not reporting a new integration wart. We are reporting what one costs inside a measurement: a documented safety property silently absent for as long as it has existed, and half of a study’s calls diverted onto a bundle-construction path that skips the corroboration cascade entirely.

6.2 Why the test suite did not help

2,712 tests passed throughout. This is not a gap in test quality but in test *shape*:

- M2 and M3 require a **second** case within one server lifetime. A unit test loads one program, asserts, and tears down. The state that goes wrong is created by the second call and cannot exist in a single-call test.
- M1 required a second *server*. The behaviour was correct against the only server exercised.
- M4 required a rich enough **input** — the composition only appears when the step budget is actually exhausted, which small fixtures never do.
- M5 was **tested and passing**. `_reconcile_with_cascade` has unit tests, and they assert exactly what it does: unresolvable patterns are dropped, missing cascade techniques are added. The function was never wrong. What was wrong was an experiment that varied the cascade’s input and attributed the output to a model downstream of it — and no test of a component can catch a misattribution made three modules away.
- M6 was **tested more carefully than anything else in the harness, and the tests were about the wrong question**. The series arithmetic had a test file of its own: averaged ranks so file order cannot break ties, an exact permutation p because four points cannot reach significance, a common-cell restriction so endpoint availability cannot pass for model size. Every test asserted something true. None asked whether the rows entering the correlation were comparable, because the answer had been assumed at the point where the arms were assembled rather than decided anywhere a test could see it.
- M7 had **nothing left to assert on**. A unit test would build the model and check that its output ceiling is 8,192 — and it is, at every level the object model exposes. The defect existed only in the serialised request, which no test in this suite inspects and which no application code reads. Testing the property as the system represents it confirms exactly the thing that is not true.

Four preconditions — a second call, a second server, a large input — are exactly what a fast unit suite is designed to avoid. The last three are worse: **M5, M6 and M7 are invisible to unit testing in principle**, because every unit involved behaved as specified. Their common shape is that the defect lived in the *composition* — which measurement was attributed to which cause, or which representation of a value actually crosses the boundary — and a test that pins a function’s behaviour cannot reach an assumption made when its inputs were chosen or when its output was serialised.

6.3 What did find them: output cardinality

Five of the seven were found by the same observation: **a number that repeated where variation was expected**. M6 and M7 were not, and §6.2 says what did find them — their outputs vary, so there is no repetition to notice.

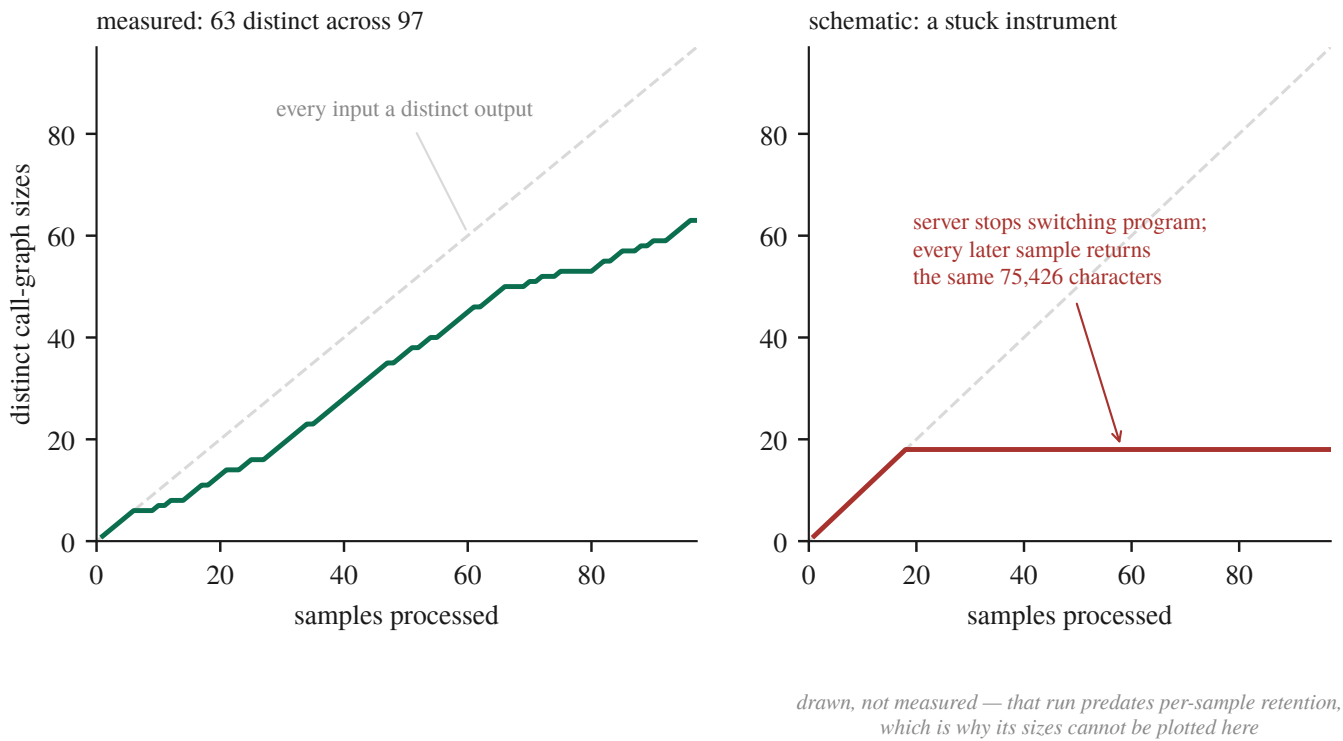


Figure 4: Figure 4: The whole detector, drawn. Plot distinct outputs against inputs processed and a healthy instrument tracks the diagonal, while a stuck one goes flat — no ground truth required, because the diagnosis is in the shape. Left is measured: 63 distinct call-graph sizes across 97 samples, the staircase’s flat treads being genuinely identical binaries rather than a fault.

Table 17: What the output-cardinality check saw and which defect each observation turned out to be. Every row is a number repeated where variation was expected.

observation	defect
a priority hint of exactly 2,575 characters for two unrelated binaries, and a third in an earlier session	M2
call graphs identical to the character (404,337 chars, 11,798 lines) for samples of 241 KB and 139 KB	M2
66 consecutive samples at exactly 75,426 characters	M3
every tool call returning the same validation error	M1
a 25-minute call that always produced one claim and zero techniques	M4
a Jaccard of 1.000 with a zero-width interval , from a model asked the same question 32 times at temperature 0	M5

M5 extends the detector in a direction worth stating, because it is where the idea is least comfortable: the repeated number was **our own result**, not a server’s response, and the variation that failed to appear was the model’s. A perfect agreement is the same signal as an identical digest, and it deserves the same suspicion.

The generalisation is one line of arithmetic: **before trusting a batch measurement, ask how many distinct outputs the N inputs produced.** If far fewer than N differ on a dimension that should vary — output length, digest, element count — the instrument is repeating itself, and repetition is what stale state looks like from outside. It needs no ground truth and no oracle, which is what makes it usable exactly where correctness is hard to check.

The right panel is a schematic, and the reason is the argument of §6.4. It depicts M3 — the run in which 66 consecutive samples returned a byte-identical 75,426-character call graph — and we cannot plot it, because that

run predates the per-sample retention policy those very failures caused us to adopt. The most important curve in this paper is the one we are unable to draw.

We now report it beside every batch result: 50 distinct call-graph sizes across 79 samples, then 63 across 97.

This is a refinement, not a discovery, and we checked before saying otherwise. “Distinct inputs should produce distinct outputs” is an ordinary metamorphic relation [22]; metamorphic testing exists precisely for programs whose correct output is unknown — the “*notorious oracle problem*” its own survey names. Stress-testing an LLM judge’s consistency under input variation is likewise established practice [23]. And the genus is described: a longitudinal study of a production LLM agent runtime defines the meta-pattern as “*a failure whose error signal never reaches a human in actionable form*” and gives a five-class taxonomy [20] into which all four of our **boundary** mechanisms fall. M5, M6 and M7 share the meta-pattern but not the setting: no server misled us, and no protocol was involved. Two arose inside our own analysis code and one inside the production pipeline, and in all three the taxonomy’s classes — every one of which describes a call to something else — do not reach.

6.4 What it cost

A completed study is withdrawn from this paper.

A seven-year, 210-sample temporal-drift analysis ran the full pipeline per sample against a long-lived server that was never restarted between samples — M2’s precondition exactly. Whether it was affected **cannot be determined**, because its per-sample outputs were not retained. The result had passed review, entered the claim ledger as novel, and was cited internally for weeks.

Two policies follow, and both are cheap:

1. **Retain per-sample outputs for every study.** The withdrawal is not caused by the defect; it is caused by the defect *plus* the inability to re-ask the question afterwards. Aggregates cannot be re-interrogated.
2. **Restart the shared server between samples.** In our sweeps this recovered 12 of 14 samples previously recorded as unmeasurable — the failures belonged to the *state they inherited*, not to the samples. Fresh state per sample removes a whole class of the failures above.

6.5 The claim we are making

Not that silent failures exist at tool boundaries — that is documented. Ours is narrower:

- three mechanisms at three **distinct** integration boundaries (argument encoding, server-side session state, HTTP status versus body), plus one from **composing** two local safety bounds;
- the setting — an **evaluation pipeline for security research**, where the artefact is not a degraded user session but a **measurement that is wrong and looks right**, and where the failure therefore propagates into a published claim rather than into a support ticket;
- a detector reported **alongside results** rather than run as a test, because the failure appears in the batch and not in any single call;
- two further failures (M5, M6) with **no server involved at all** — an ablation that varied a deterministic code path and reported a language model, and a correlation that ranked a configuration flag and reported a parameter count. Both had passing unit tests over the exact functions concerned. They extend the claim from “other people’s servers can mislead you” to *the same shape occurs wherever a measurement’s cause is assigned rather than measured*;
- and one (M7) in the **production pipeline**, where a documented safety property — a bounded verdict generation — was configured correctly at every level the code can inspect and removed entirely by a parameter rename during serialisation. It is the case that shows the shape is not peculiar to evaluation: the artefact the analyst receives was affected, not a number in a table;
- and a demonstrated price rather than a hypothesised one.

In a pipeline assembled from other people’s servers, the correctness of the model is not the binding constraint on the correctness of the result. **A server that answers is not the same as a server that answered your question.**

7 Discussion

Interpretation lives here. Results states what was measured; this section says what follows from it, and separating the two is not a formality — several of the sentences below were sitting in Results, where a reader could not tell a measurement from a reading of one.

7.1 What a null means at this cluster count

The most consequential thing we can say about our own negative results is that most of them are bounds rather than equivalences, and that we did not know this when we first wrote them down.

Five of the studies reported here run on a corpus of five samples. At five clusters the exact two-sided cluster permutation test cannot return a value below 0.0625, so **no comparison measured on that corpus can reach $\alpha = 0.05$ whatever its effect size**. That is a property of the design, not of the data, and no amount of extra decoding repeats moves it — repeats add rows inside clusters, and rows inside clusters are not what the test counts. Multiplicity correction is therefore beside the point on that corpus: we apply Benjamini-Hochberg over two declared families and it finds nothing to correct, because there was nothing at $\alpha = 0.05$ to begin with.

The practical consequence is that a null has to be reported with its resolution. Our frontier-model comparison could detect a difference of 0.300 F1 at 80% power and returned +0.0026. Both facts are true, and only the pair of them is informative: the second alone reads as equivalence, and the first alone reads as a failed experiment. We now report the minimum detectable effect beside every null in this paper, and we would put it beside every null we read in someone else’s.

This also changes what we think the cheap fix is. The instinct on seeing an underpowered paired design is to add repeats, because repeats are cheap and samples are expensive. On clustered data that instinct is exactly wrong. Adding a sixth fixture would have moved the attainable p from 0.0625 to half of it; adding a sixth repeat to five fixtures moves it not at all.

7.2 Where a measurement’s cause is assigned rather than measured

The failure taxonomy we counter-searched against organises silent failures by the boundary crossed [20]. Four of our seven crossed one: a sandbox, a decompiler server, an inference endpoint. Three crossed none. No server misled us, no protocol was involved, and no network was in the path.

What the seven share is not a boundary but an **attribution**. In each case a measurement’s cause was assigned rather than measured. An ablation varied a deterministic post-processing step and we attributed the difference to a language model. A rank correlation ordered a reasoning-configuration flag and we attributed the ordering to parameter count. A documented output ceiling was renamed during serialisation and we attributed boundedness to a component that had never once been bounded. In every case the number was real and the arrow pointing to its cause was drawn by us.

We offer that as a proposal rather than a taxonomy. Seven cases from one project is not a class, and the published taxonomy is built on a corpus we cannot match. But it does suggest a different question to ask of an evaluation than the one boundary-oriented thinking suggests: not *did any component fail silently*, but *for each number I am about to report, did I measure its cause or assign it*.

The correction described in §5 is that question turned on ourselves and answered badly. Every interval in an earlier draft of this paper attributed its width to sampling variation among independent observations. The observations were not independent, the width was an artefact of the unit we resampled at, and nothing in the pipeline, the tests or the review caught it — because the defect was never in a function.

7.3 What a practitioner should take from the negatives

We are wary of drawing implementation advice from one pipeline on one machine, so this is narrow.

Measure the firing rate before the effect. A mechanism that engages on 0.82% of its inputs produces an ablation whose null describes the cases where it never ran. This is the cheapest practice in the paper and it changed which experiments were worth running rather than merely how they were reported.

Count the distinct outputs. Four of our seven defects were found by asking how many distinct outputs N inputs produced. It needs no ground truth and no oracle, which is what makes it usable where correctness is hard to check. It belongs beside the sample size in an evaluation report rather than inside a test, because the failures it catches need a second call in one server lifetime and a unit test writes one.

Retain per-sample outputs, and then ask what else the study is made of. We adopted per-sample retention after losing a study to its absence, and the rule was scoped to the object under study. The next failure was in the environment: four arms of a paired ablation died on host memory, we had their outputs and not the host state, and they remain unattributable. The rule you write after a failure is scoped to that failure.

A server that answers is not the same as a server that answered your question. Four of our defects are instances of it, and the class is documented [20]. Our contribution is the setting and the price: in an evaluation pipeline the artefact of such a failure is not a degraded user session but a measurement that is wrong and looks right, and the price we paid was a completed 210-sample study that can no longer be asked its question.

7.4 What we are not in a position to say

We cannot say that multi-agent decomposition does not help. We can say that on five samples, at equal token budget, we could not detect a difference larger than 0.223 F1, and did not.

We cannot say that model capacity is not the ceiling. We can say that a 3.4× larger model did not separate from ours at a resolution of 0.300 F1, that the two arms differed in a configuration flag worth more than any architectural effect we measured, and that the endpoint would not let us remove the difference.

We cannot say that verbal confidence is uninformative in general. We can say that in this pipeline, on this corpus, it ranks correct claims above incorrect ones at AUC 0.550 with an interval of [0.475, 0.671] that contains chance, and that 3 of 5 samples sit at or below chance individually — so the deterministic gates keyed to that number are keyed to something we cannot distinguish from noise.

And we cannot say that our seven mechanisms generalise. What we can say is what each of them cost.

8 Threats to Validity

We organise this section around the nine pitfalls of *Chasing Shadows* [9], which surveyed all 72 peer-reviewed LLM-security papers of 2023–2024 and found every one exposed to at least one, with only 15.7% discussing any. We add two checks the survey does not have, both of which caught errors in this work. Where a pitfall is genuinely closed we say so; where it is not, we say what remains rather than what we intend.

8.1 The threat that actually materialised

Most threats sections describe hazards. This one begins with damage, because reporting it is the paper’s main methodological point.

One study is withdrawn. A seven-year, 210-sample temporal-drift analysis was completed, entered our claim ledger as a novel result, and is now retracted. It drove the full pipeline per sample against a long-lived Ghidra server that was never restarted between samples — which is precisely the precondition for a defect we later found, in which a loaded program never becomes the server’s *current* program, so every sample after the first is described using the **first sample’s binary** (§3.14). Whether that run was affected cannot be determined: its per-sample outputs were not retained. The result is therefore not restated anywhere in this paper, and the axis it measured — input-era drift,

as distinct from train/test temporal misalignment — is presented as an open question with a corrected instrument required before any bound is claimed.

Three defects of this kind were found in a single day, all sharing one shape: **a plausible wrong answer with no error anywhere**.

Table 18: The three boundary mechanisms that reached published results, with what each affected and how it was found.

	mechanism	what it silently produced
M1	unset optional MCP arguments were sent as null	all 36 sandbox tools refused; the pipeline saw only empty results
M2	load_program reported success <i>with the right program name</i> but did not change the server’s current program	every sample after the first analysed the first binary
M3	a refused load answers HTTP 200 with an error body	hints and function hashes built from a different binary

None was caught by a test suite of **2,712 passing tests** — not by accident: M2 and M3 require a *second* case within one server lifetime, and a unit test writes one. A fourth defect, found later in the same week, made the analyst return **zero techniques on any binary rich enough to exhaust its step budget**, because two protective bounds composed: exceeding the step cap triggers a synthesis call that received a *fresh* copy of the full time budget, so loop-plus-synthesis overran the hard cap by construction (§3.15).

Three further defects were found later, and none of them involved a server. An ablation varied a deterministic post-processing step and we attributed its output to a language model; a rank correlation over model size ranked a reasoning-configuration flag instead and recovered the published scaling coefficient almost exactly; and the verdict model’s documented 8,192-token output ceiling was renamed by our client library during serialisation to a key the inference server does not read, so a component described as bounded had never once been bounded. These three had passing unit tests over the exact functions concerned, which the four above did not — the defect was never in a function, but in which measurement was attributed to which cause, or in which representation of a value crossed a boundary. §6 reports all seven with the layer each occurred at.

We report these because we believe the interesting threat to an LLM-plus-tooling pipeline is not that the model is wrong. It is that **the instrument answers a different question than the one asked, convincingly**.

8.2 The nine pitfalls

P1 Data poisoning — CLEAR. Nothing is trained. Retrieval corpora are curated and their provenance is documented.

P2 Label inaccuracy — CLEAR. Ground truth is MITRE-curated, and **no LLM scored any result in this work**. This was a standing constraint, not an outcome: an internal readability assessment was performed with an LLM as a development aid and is deliberately excluded from this paper, so that the absence of LLM-as-a-judge scoring remains unqualified. A consequence is stated under E.7: no human analyst rated any report either.

P3 Data leakage — PARTIAL. One instance was found by us, disclosed and mitigated. It was never systematically audited, and the pitfall’s own recommendation applies: **the model’s training data is unknown to us and memorisation was not probed**. We acknowledge this rather than argue around it.

P4 Model collapse — CLEAR. Augmenting the long-term memory corpus with LLM-generated cases was considered and refused, with a cited rationale.

P5 Spurious correlations — PARTIAL. Two were found and are reported. No systematic perturbation testing was performed.

P6 Context truncation — EXPOSED. Truncation is designed-in throughout — chunking at function boundaries, a per-call evidence cap, a 40-step ReAct budget, an 8,192-token verdict cap, a 400-char hint cap — and the counters

now exist to report frequency, with pass-throughs counted so the rate has a correct denominator. What we cannot yet report is the distribution over a full cohort. Two specific findings belong here:

- The enumeration of truncation sites was **itself incomplete**. A 20,000-edge cap on the call-graph fetch was discovered only while instrumenting a different measurement, and it silently truncates 1 of 97 samples (§3.15). We therefore describe how our list was built rather than presenting it as exhaustive.
- Bounds **compose**. Exceeding the step cap guarantees an attempt at the time cap, and on rich binaries that attempt could not finish — two limits producing an empty result between them. This is §1.7.1’s shape in a new place: there, removing a hint made the judge overrun a 600 s ceiling and return an empty bundle 6/17 times.

P7 Prompt sensitivity — PARTIAL. A structured prompt-variation study exists. Production prompts are fixed and were not varied per model. Related: our own instrument-validity failure, in which a single-run parsed-claim count ranked the same arms differently at different decoding budgets, is prompt sensitivity surfacing as measurement error.

P8 Surrogate fallacy — PARTIAL, on evidence. Every local result comes from **one model on one machine**: Qwen3.6-35B-A3B at IQ3_K_R4, ik_llama.cpp, single host. Four claims were rescoped to that exact identifier. A second endpoint has now run to completion — a 120B reasoning model on the same fixtures, repeats, prompt and token budget — and **did not separate from the local model**: paired $\Delta F1 +0.003$, 95% CI $[-0.077, +0.081]$, $n=25$, better on 12 and worse on 13 (§3.16). A 3.4× parameter advantage produces no measurable difference here, which is evidence *against* the pitfall’s usual worry rather than merely an absence of evidence for it.

An interim version of this arm reported 0.5025 at $n=9$ and read as a lead; completing the sample moved the estimate through the local mean and out the other side. We record that because the arm had been truncated by a daily request quota, not by anything about the samples — the failure mode is reading a difference off an underpowered arm, and this audit carried the wrong number for a day.

That null is confounded, and the confound cannot be removed on that endpoint. The local arm runs with its reasoning stream disabled; the 120B arm ran with reasoning on, spending 56.5% of its output budget there. We re-ran it with the flag set: the provider accepted the parameter and ignored it — 56.2% reasoning, F1 0.4149 against the original 0.4162 (paired $\Delta -0.0014$, CI $[-0.0929, +0.0874]$). The re-run replicates the arm rather than correcting it, and no further re-run will do better, because the control is not exposed. We therefore report the null with its confound named. The size of what is being confounded is not small: on two separate models the same flag is worth **0.34 and 0.45 F1**, larger than every architectural effect in this paper combined.

One quantisation is no longer an unmeasured threat. The model we run locally is also hosted by its vendor at full precision, on an endpoint that does honour the reasoning flag. Paired on the same fixtures and repeats, our 3-bit IQ3_K_R4 deployment scores **0.0629 higher** than the vendor’s hosting of the same weights (95% CI $[-0.1484, +0.0256]$, $n=25$) — an interval containing zero, and pointing away from the direction the threat assumes. Two serving stacks differ alongside the precision, so this bounds *the deployment* rather than quantisation alone; but the specific worry that a 3-bit local model understates what these architectures can do is not supported.

What P8 cannot close is the size series, and the reason is measured rather than budgetary. Testing arXiv:2606.18166’s parameter-size trend needs three arms at three sizes, matched on the flag that outweighs size. Of the endpoints available, the one that honours the flag hosts the same 35B model we already run, and the one above 35B does not honour it. Two matched arms exist and both are 35B. We state this as a limitation rather than reporting a correlation over unmatched arms — which, run once before this constraint was enforced, returned $\rho=+0.866$ and would have agreed with the literature for the wrong reason.

What also remains is **coverage, not power**: the second model has been measured on five synthetic fixtures, not on the malware cohort, where evidence bundles are longer and messier. That is the arm the free tier’s 50 requests/day makes a two-day run or a paid one.

P9 Model ambiguity — PARTIAL. Model and engine are both pinned: GGUF digest, HuggingFace revision, retrieval date, quantiser, **and the imatrix calibration dataset** — which most papers reporting a quantisation level cannot name. The engine commit was recovered and *proved* to describe the build (identical 837-file source list; exactly one generated file differs). It is PARTIAL rather than CLEAR for a reason worth stating: **the running binary**

reports its version as unknown, and the commit was recovered from a vendored copy of the sources rather than from the artifact. Build provenance must be captured at build time; ours was reconstructed, and we say so.

8.3 Two checks the survey does not include

Both were added after they caught errors here.

The tenth check — diff the claim register against the evidence log. All nine pitfalls concern the relationship between claims and experiments. Two failures sat one level below that. A claim was recorded as novel one hour after our own findings log had described it as a replication; and this audit mis-stated one of its own rows, attaching a model caveat to a result produced by a server-free harness. Neither needed a literature search — only reading two of our own documents against each other. Applied again later, it found **three ledger rows that had drifted from the evidence log**, including one still carrying an interim figure after the final measurement had landed. A project disciplined enough to keep both a findings log and a claim ledger has, by that fact, created the conditions for the two to diverge.

The eleventh check — count the distinct outputs. Before trusting a batch measurement, ask how many distinct outputs the N inputs produced. If far fewer than N differ on a dimension that should vary — length, digest, element count — the instrument is repeating itself, and repetition is what a stale-state bug looks like from outside. All three boundary defects above were caught this way: a 2,575-char hint repeated across unrelated samples; call graphs identical to the character for binaries of 241 KB and 139 KB; 66 consecutive samples at exactly 75,426 characters. It costs one line of code and needs no ground truth. We now report it alongside results (50 distinct call-graph sizes across 79 samples; 63 across 97).

This check is a **refinement, not a discovery**, and we counter-searched it before claiming otherwise: “distinct inputs should produce distinct outputs” is an ordinary metamorphic relation [22], stress-testing a judge’s consistency under input variation is established practice [23], and the genus — silent failures in production LLM agent runtimes, defined as “*a failure whose error signal never reaches a human in actionable form*” — is described with a five-class taxonomy our defects fall into [20]. What we add is the setting: an evaluation pipeline for security research, where the output is not a degraded user session but **a measurement that is wrong and looks right**, together with three mechanisms at three integration boundaries and one withdrawn study as the demonstrated cost.

The taxonomy’s five classes each describe a call to something else, and three of our seven crossed no boundary at all. They share its meta-pattern and sit outside every class it offers, which is why we suggest — as a proposal, on seven cases from one project — that the organising principle covering both is **attribution rather than boundary**: the shape appears wherever a measurement’s cause is assigned rather than measured.

8.4 Construct validity: what the ground truth can and cannot support

Per-sample ground truth is **family-level MITRE uses**, which is coarse: a single Emotet binary need not exhibit all ~47 techniques catalogued for Emotet, and a packed sample exhibits fewer still. Absolute recall therefore carries a structural ceiling and precision carries noise. Two consequences we accept rather than argue with:

1. Absolute F1 values in this paper are **not comparable to work using per-sample expert labels**.
2. The bias is approximately constant across arms, so within-study contrasts are the defensible reading — which is exactly why a baseline matters. **The sandbox alone, with no LLM anywhere, scores F1 0.153 [0.134, 0.171]** on our cohort of 97. Every pipeline figure is read against that rather than against zero, and on the samples where both have run the full pipeline is 0.003 F1 above it.

8.5 The dynamic channel is two-thirds constant

Any claim about what the sandbox contributes has to survive one measurement of what the sandbox actually reports. Across the 97 archived analyses there are **143 distinct network domains**, and **38 of them appear in every single**

sample — the analysis VM’s own vendor telemetry, present in each capture whatever was detonated. Per sample, between **55.9%** and **79.2%** of observed domains are cohort-ubiquitous, median **67.9%**.

So two-thirds of the network evidence in a “dynamic” arm is the instrument describing itself. A treatment that constant is weaker than its name suggests, and an effect attributed to malware behaviour may be partly an effect of the VM’s idle traffic. We report the ubiquitous share alongside every dynamic-versus-static contrast rather than in a footnote, because the contrast cannot be read without it.

The same measurement bounds two Layer-0 sources from the other direction. The LOLBin layer and the DGA layer produce a claim on **0 of 97** samples while being fed a median of 8888 recorded API calls and 48–68 domains respectively — they are offered the evidence and decline it. That is why neither appears as an arm in the cascade ablation: giving a mechanism an equal share of ground truth when it never engages in the deployment would measure a system that does not exist.

An earlier version of this measurement, on the 43 analyses that survived before the cohort was recovered, gave 130 domains with 40 ubiquitous and a median share of 71.4%. More than doubling the cohort moved the figure by three points and left the conclusion where it was.

8.6 The host as an uncontrolled variable

A threat we did not anticipate and met late: **the machine a measurement runs on is part of the instrument**. Our working set — model server ~16 GB, analysis worker ~8 GB, disassembly container up to 6 GB — does not fit a 31 GiB host alongside an interactive session. During a paired ablation the swap file was exhausted and the model server had 2.3 GB of its own address space paged out; an arm then exceeded a 594-second budget on a prompt trimmed to 16,000 characters. Whether that arm failed because of the pipeline or because of the host **cannot be determined**, because we recorded the pipeline’s outputs and not the host’s state.

The affected arms are reported as `unattributable` rather than as failures, and the ablation is reported as incomplete rather than as a null. We flag three things a reader should take from this rather than from our fix:

1. **Wall-clock bounds are host-dependent measurements wearing the costume of a system property.** Any result in this paper that turns on a timeout — the salvage-path failure of §6, the timing column of §7 — is a statement about this pipeline *on this machine under whatever load it had*.
2. **The retention rule we adopted after the first failure did not cover this one.** §4.5 says to retain per-sample outputs, and we did. The rule was written about the object under study, and what went unrecorded was the environment the study ran in.
3. **A screen for this can only be applied forward.** Arms already collected without host state cannot be re-screened, which is the same structural problem that withdrew our drift study, in a new variable.

A postscript, because it took three attempts to see. Two of the interruptions above we attributed to memory pressure and treated with progressively better limits: a floor, then a strike counter, then an allowance for declared allocations, then marking the model server as the kernel’s preferred OOM victim. All four were correct and none was the cause. The harness had started the server as a **child process**, so it ran inside the cgroup of the terminal that launched it and its memory was accounted to that scope; the kernel’s log named the scope plainly and we did not read it until the third failure. Every fix we made chose *which process* the kernel would kill, and the defect was *whose accounting it was killed inside*.

We report this because it is the same error as M1–M4 in a different register: a mechanism that looked like it was working, an intervention that addressed a real thing at the wrong layer, and a log line that had contained the answer for two days. A measurement environment is not a backdrop to the measurement, and the discipline that catches this is not cleverness — it is reading the error message that was already there.

8.7 Scope

Unless a statement says otherwise it was produced on one model, one machine, one quantisation and one engine build, with the identifiers in the reproducibility appendix. Where a result concerns retrieval or the deterministic cascade rather than generation, no model was involved and the binding limits are the index and the corpus instead; those are marked at the point of claim.

9 Conclusion

We built a multi-agent LLM pipeline for mapping malware evidence to ATT&CK techniques, and then spent longer measuring it than building it. This paper reports what the measurements said, and the methodological problem we kept running into while taking them.

9.1 The architecture did not survive its own evaluation

Four claims motivated the design. Measured against simpler alternatives at equal token budget, they did not hold.

Table 19: Four architectural claims against what the measurements support. Every interval resamples the unit the observations are independent at; the minimum detectable effect says what the design could have seen.

claim	measured	this design could detect
negotiated multi-agent consensus beats a single judge	$\Delta F1$ -0.0161 , 95% cluster CI $[-0.1247, +0.0819]$, at 3.2 $\times$ the tokens	0.223 F1
a multi-layer corroboration cascade improves the verdict	corroboration never reaches the artefact — and the model is not the reason: reconciliation restores every cascade technique the judge omits, and on 0 of 8 samples the judge added nothing of its own	a per-sample rate of 0.375
two-tier attribution identifies families	opcode-hash tier fires on 0 of 18 samples; family-feature retrieval contributes +0.0029 F1	—
falsification-before-confidence disciplines claims	the cap fires on 0.82% of techniques	—

None of these components is broken. Some work in isolation, and each was built for a reason we would give again. They do not move the end-to-end result. We report that as a result rather than as a tuning opportunity.

What we may not say about those nulls, and said in an earlier draft. We wrote that the noise control separated cleanly and that the nulls therefore came from an instrument shown able to detect a difference. The control does move — -0.0770 , $[-0.1337, -0.0233]$, all 5 samples in the same direction — but the effect is smaller than the design’s own minimum detectable effect of 0.120, and the exact cluster permutation test returns 0.0625, which at 5 clusters is the smallest value the test can return. No comparison on that corpus can reach $\alpha = 0.05$ whatever its effect size. The nulls stand as nulls; the sensitivity argument does not stand, and the honest reading of every one of them is a bound rather than an equivalence.

Two further negatives constrain how the rest should be read. Verbal confidence — the number the cascade and every deterministic gate consume — ranks correct above incorrect claims at **AUC 0.550**, but the interval that resamples samples rather than claims is $[0.475, 0.671]$ and contains chance; 3 of the 5 samples are at or below it, and the pooled figure is carried by the one that reaches 0.778. Verbal confidence is **indistinguishable from chance here**, which is a stronger statement than the one we set out to make and a worse one for the gates that consume it. And

a 120B reasoning frontier model did not separate from our local 35B on identical fixtures at equal *nominal* budget — paired $\Delta F1$ **+0.0026**, 95% cluster CI $[-0.1322, +0.1460]$, better on 12 and worse on 13 — at a design resolution of 0.300 F1, which is coarser than every architectural effect in this paper. That null means the design could not see a difference, not that there is none. It also carries a confound we could not remove: the frontier arm reasoned and the local one did not, and the endpoint accepts the parameter that would have matched them without acting on it. We report the confound’s size rather than its absence — on two other models the same flag is worth **+0.3427 and +0.4501 F1** — and note that we cannot show the local model to be the weaker deployment either: against its vendor’s own full-precision hosting the paired difference is -0.0629 , $[-0.2133, +0.0963]$, an interval wide enough to admit a substantial penalty in either direction.

9.2 The nulls are interpretable only because firing rates came first

A mechanism that never runs produces an ablation whose null describes the cases where it never ran. The confidence cap fires on 0.82% of techniques and the sink-reachability hint on 56.7% of samples; an ablation of the first would have been uninterpretable and reported as evidence anyway. Measuring **firing rate before effect** changed which experiments were worth running, and it is the cheapest practice in this paper.

9.3 The instrument was repeatedly wrong in ways that looked like data

This is the finding we did not expect to be the paper. Four defects at the boundaries between our pipeline and the servers it depends on each produced *plausible results rather than errors*: an unset optional argument arriving as an explicit null; a program reported as loaded, by name, that never became the one being analysed; a refused load returning HTTP 200 with the error in the body; and two protective bounds composing into an empty result. **A suite of 2,712 passing tests caught none of them**, and not because the tests were bad — each defect requires a second call, a second server, or a large input, which is precisely what a fast unit suite is built to avoid.

All four were found by one arithmetic check: **how many distinct outputs did N inputs produce?** A priority hint of exactly 2,575 characters for unrelated binaries; 66 consecutive samples at exactly 75,426. It needs no ground truth and no oracle, which is what makes it usable where correctness is hard to check, and it belongs beside sample size in an evaluation report rather than inside a test.

Three more followed, and none of them was at a boundary. An ablation varied a deterministic post-processing step and we attributed its output to a language model. A rank correlation over model size ranked a reasoning-configuration flag instead, and recovered the published scaling coefficient almost exactly — agreeing with the literature, which is the direction a wrong result is least likely to be questioned. And the verdict model’s documented output ceiling was renamed by our client library during serialisation to a key the inference server does not read, so a component described as bounded had never once been bounded; one call was measured at 30,155 generated tokens against an 8,192 cap. The cardinality check does not find these — their outputs vary. Each had passing unit tests over the exact function concerned, because the defect was never in a function.

We are careful about what is new here, and we counter-searched before claiming. Silent failure at tool boundaries is documented, the metamorphic relation behind our detector is ordinary, the genus has a published taxonomy, that an inference-time constraint can invert model rankings is established with a 28.4-point reversal [24], and the token-limit incompatibility is in the servers’ own issue trackers. Our contribution is narrower: three distinct integration boundaries plus one composition of local bounds, in a setting where the artefact is **a measurement that is wrong and looks right**; a selection rule that follows from a provider accepting a parameter and not acting on it, so arms must be matched on their measured configuration rather than their requested one; and a demonstrated price rather than a hypothesised one.

One observation we offer as a proposal rather than a result. The published taxonomy organises these failures by the boundary crossed, and three of ours crossed none — no server misled us, and no protocol was involved. What the seven share is not a boundary but an attribution: in each case a measurement’s cause was assigned rather than measured.

9.4 What it cost, including while writing this

One completed study is withdrawn — a 210-sample temporal-drift analysis that had passed review and been cited internally for weeks. It is withdrawn not because the defect certainly affected it, but because its per-sample outputs were not retained and **the question can no longer be asked of it**.

The discipline then caught us a second time, one level up, during the final experiment. A paired ablation halted at 10 of 24 arms when the host exhausted its swap file; four arms had died in ways that may belong to the pipeline or to a machine whose model server had 2.3 GB of itself paged out. We had retained per-sample outputs, as our own rule requires. We had not retained per-sample **host state**, and that is what the question needed. The arms are reported as unattributable rather than as failures, and the effect of that mechanism remains unmeasured.

We report this rather than quietly re-running it because it is the paper’s own thesis arriving uninvited: the rule you wrote after the last failure is scoped to the last failure. Retention policies are written about the object under study; the environment the study runs in is also part of the instrument.

9.5 What we claim, and what we do not

We claim: the measured negatives above; seven failure mechanisms with the layer each occurred at, four of them at a boundary with another server and three inside our own code; a detector that found the four boundary cases and costs one line, together with what found the other three; and a no-LLM baseline of **F1 0.1526** [+0.1114, +0.1998], n=97, without which none of our F1 numbers refer to anything — and against which the full pipeline gains **+0.0030 F1**.

We claim one result about the architecture that we did not go looking for: the verdict model’s influence on the artefact is **conditional on the model working**. Where it returns a parsable verdict it supplies 0 of 99 techniques, because a post-processing step restores the deterministic set regardless; where it fails, a text fallback lifts identifiers out of the unparseable response and 47 techniques reach the analyst that no evidence source corroborated. The design suppresses the model’s contribution exactly when the model is functioning and admits it exactly when it is not.

We do not claim that multi-agent decomposition cannot work, that these retrieval designs are worthless, or that our practices generalise beyond one system. This is one pipeline, one 35B model at one quantisation, on one machine, evaluated on Windows PE malware. The dynamic-evidence experiments this design most needs remain gated behind sandbox access and are not reported.

What we do assert is a change of default. In a pipeline assembled from other people’s servers, the correctness of the model is not the binding constraint on the correctness of the result. Long before a benchmark score is a claim about a system, it is a claim about an instrument — and **a server that answers is not the same as a server that answered your question**.

Declarations

9.6 Ethics and containment

Every sample analysed in this work is live Windows PE malware. The 97-sample cohort and the 210 dated samples of the withdrawn drift study were obtained from MalwareBazaar (abuse.ch) under its terms of use, which permit research redistribution of samples but not of the corpus as a package.

Containment. Detonation happens only inside a CAPE sandbox on a dedicated Windows guest image, reverted to a clean snapshot between analyses. The sandbox host is reachable from one network segment and from no public route; it holds no credentials for any other system, and no analysis machine shares a filesystem with the host that runs the pipeline. Binaries are held only in a directory excluded from the host’s real-time scanner, are never committed to version control, and are not redistributed with this paper. Network traffic generated during detonation leaves the guest through that one segment and is captured; it is not proxied to any third-party service.

We note a limitation of that arrangement rather than claiming it is sufficient: the guest reaches the public internet during detonation, because a sample that cannot resolve its command-and-control infrastructure produces different

behaviour from one that can, and the network evidence channel would otherwise be empty for every sample. Live detonation with egress is the standard arrangement for behavioural sandboxing and it is the arrangement whose consequences we report, including that the domains every sample in the cohort contacted turn out to be the guest image describing itself rather than anything the samples did.

No human subjects. No user study was conducted, no personal data was collected, and no human analyst scored any output. That last point is a limitation of the evaluation, stated as such in Threats to Validity, not a claim about ethics.

9.7 Data availability

The evaluation harnesses, the derivation of every number in this paper, the per-sample records they read, and the scripts that build the figures and assemble the document are released with the paper. The archive contains:

- every per-sample record retained by every study reported here, in the JSON the harness wrote;
- the offline re-analysis that recomputes every interval, design effect, minimum detectable effect and adjusted p-value from those records, with its random seed;
- the ATT&CK ground-truth fixtures, derived from the public `mitre-attack/attack-stix-data` repository under the ATT&CK Terms of Use;
- the model digest and inference-engine commit needed to reproduce the local arm.

Withheld, and why. Malware binaries are not released; they are obtainable from the sources named in Background by their SHA-256 digests, which are in the archive. Sandbox reports are not released in full because they embed strings and memory contents from the samples. The sandbox host’s address is withheld; it appears nowhere in the archive and is referred to throughout as a private address. One dataset used for retrieval-corpus construction is redistributed by its authors under terms that do not permit our redistribution, and is referenced rather than included.

Not available. The 210-sample temporal-drift study’s per-sample outputs do not exist. They were written to a path that was valid on the machine the harness was first written on and silently relative on the machine it ran on; the study is withdrawn on that account and the withdrawal is reported in Results rather than quietly repaired. The manifest of the 210 samples survives and is included, so the cohort can be reconstructed even though the results cannot.

9.8 Reproducibility

Two runs of the offline analysis over the released records produce byte-identical output, and this is asserted by a test in the released suite rather than claimed here. Every interval carries the seed, the iteration count and the number of clusters it was computed from, in the artifact itself, so an interval cannot be read without the information needed to reproduce it.

The live arms are not bit-reproducible and we do not claim they are. Decoding is at temperature 0.1 on the local arm and at each provider’s default on the hosted ones, one of which accepts a reasoning-configuration parameter and does not act on it — a fact we established by measurement and report as a result. Anyone re-running the live arms should expect the numbers to move and should read the intervals accordingly.

9.9 Use of generative AI

Large language models are the object of study in this work: the pipeline under evaluation is built from them, and every measurement reported here is a measurement of their behaviour in it.

Separately from that, generative AI was used as an authoring and engineering aid — drafting and revising prose, writing evaluation harnesses and their tests, and performing literature searches whose results were then verified by fetching each source. The verification is not incidental to this disclosure: of the nine citations added by those searches and checked one by one, **three said something the source did not**. One was quoted for a sentence absent from the

paper, one was cited for a practice it does not describe, and one blended two incompatible published accounts into a single attribution. All three are corrected in the text and the corrections are recorded, because a search summary that reads as a citation is the same class of failure this paper is about.

An internal readability assessment of report narratives was performed with a language model as a development aid. Its results are deliberately excluded from this paper and no LLM was used to score any result reported here.

The authors take responsibility for all content, including every number, every citation, and every claim about what the measurements support.

9.10 Competing interests

The authors declare no competing financial or non-financial interests. No external funding supported this work. All measurements were taken on hardware owned by the authors, except for calls to three commercially hosted inference endpoints, which were paid for at their public list prices and whose providers had no involvement in, or advance knowledge of, this evaluation.

References

Verification status is recorded per entry rather than assumed. Of the nine sources added by the adjacent-field counter-search and checked individually, three had been cited for something they do not say; those are corrected in the text and marked here. Sources listed in an earlier draft as company for the ones actually fetched have been dropped rather than carried as decoration.

- [1] W. Yu et al. “Maltracker: A Fine-Grained NPM Malware Tracker Copiloted by LLM-Enhanced Dataset.” *ISSTA 2024*. DOI 10.1145/3650212.3680397.
- [2] A. Rollinson and N. Polatidis. “LLM-Generated Samples for Android Malware Detection.” *Digital* 6(1):5, 2026. DOI 10.3390/digital6010005.
- [3] W. Zhao et al. “AppPoet: LLM-based Android malware detection via multi-view prompt engineering.” *Expert Systems with Applications* 262, 2025. arXiv:2404.18816.
- [4] S. Saha et al. “MaLAWare: Automating the Comprehension of Malicious Software Behaviours using LLMs.” *MSR 2025*. arXiv:2504.01145.
- [5] M. Büchel, T. Paladini, S. Longari, M. Carminati, S. Zanero, et al. “SoK: Automated TTP Extraction from CTI Reports — Are We There Yet?” *USENIX Security 2025*, pp. 4621–4641.
- [6] K. D’Oosterlinck, O. Khattab, F. Remy, T. Demeester, C. Develder, and C. Potts. “In-Context Learning for Extreme Multi-Label Classification.” arXiv:2401.12178.
- [7] A. Lekssays, U. Shukla, H. T. Sencar, and M. Parvez. “TechniqueRAG: Retrieval Augmented Generation for Adversarial Technique Annotation in CTI Text.” *ACL Findings 2025*. arXiv:2505.11988.
- [8] “Identifying Adversary Tactics and Techniques in Malware Binaries with an LLM Agent” (TTPDetect). arXiv:2602.06325.
- [9] R. Evertz, N. Risse, C. Neuer, N. Müller, S. Normann, et al. “Chasing Shadows: Pitfalls in LLM Security Research.” *NDSS 2026*. arXiv:2512.09549. Living appendix at llmpitfalls.org.
- [10] B. Bertalaníč and B. Fortuna. “The Cost of Consensus: Isolated Self-Correction Prevails Over Unguided Homogeneous Multi-Agent Debate.” arXiv:2605.00914.
- [11] D. Tran and D. Kiela. “Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets.” arXiv:2604.02460.
- [12] D. Lea, J. Ghawaly, G. G. Richard III, A. Ali-Gombe, and A. Case. “REx86: A Local Large Language Model for Assisting in x86 Assembly Reverse Engineering.” *ACSAC 2025*. arXiv:2510.20975.
- [13] J. Ng and A. Milani Fard. “Evaluating Retrieval-Augmented Generation for Explainable Malware Analysis.” Poster, *ACM SecDev 2026*. arXiv:2605.03140.

- [14] R. B. Metz, N. Spolaôr, E. A. Cherman, and M. C. Monard. “Comparing published multi-label classifier performance measures to the ones obtained by a simple multi-label baseline classifier.” arXiv:1503.06952.
- [15] L. Lange, H. Adel, et al. “AnnoCTR: A Dataset for Detecting and Linking Entities, Tactics, and Techniques in Cyber Threat Reports.” *LREC-COLING 2024*. arXiv:2404.07765. CC-BY-SA 4.0.
- [16] M. Abdallah, J. Holdcroft, R. Ali, and A. Jatowt. “Are LLM-Based Retrievers Worth Their Cost?” arXiv:2604.03676.
- [17] H. T. Ng et al. “The CoNLL-2014 Shared Task on Grammatical Error Correction.” *CoNLL 2014*; C. Bryant, M. Felice, Ø. E. Andersen, and T. Briscoe. “The BEA-2019 Shared Task on Grammatical Error Correction.” *BEA 2019*. — cited for the F0.5 convention, which weights precision twice recall to penalise over-correction. *Re-anchored 2026-08-15: two candidate papers previously listed here were never fetched and are dropped; the convention is documented by the shared tasks themselves.*
- [18] S. Welleck, I. Kulikov, S. Roller, E. Dinan, K. Cho, and J. Weston. “Neural Text Generation with Unlikelihood Training.” arXiv:1908.04319; H. Li, Z. Lan, et al. “Repetition In Repetition Out: Towards Understanding Neural Text Degeneration from the Data Perspective.” arXiv:2310.10226. — *Corrected 2026-08-15. An earlier draft attributed a single blended account to these two. Welleck et al. blame the likelihood objective itself; the training-data account is the second paper’s, which argues self-reinforcement explanations reduce to it.*
- [19] A. Lazaridou, A. Kuncoro, E. Gribovskaya, et al. “Mind the Gap: Assessing Temporal Generalization in Neural Language Models.” *NeurIPS 2021*. arXiv:2102.01951.
- [20] W. Wu. “When Errors Become Narratives: A Longitudinal Taxonomy of Silent Failures in a Production LLM Agent Runtime.” arXiv:2606.14589. — *Abstract read; full text not read. Our mapping of M1–M3 onto its five classes is our reading of the abstract and is stated as such in the text.*
- [21] “From REST to MCP: An Empirical Study of API Wrapping and Automated Server Generation for LLM Agents.” arXiv:2507.16044. — *Corrected 2026-08-15. A sentence previously quoted from this paper is not in it; it came from a search summary. What the abstract does state — 76% of sampled tools wrap successfully, 94.2% after automated repair — is what is cited now.*
- [22] Y. Li, Z. Liu, P.-L. Poon, D. Towey, C.-A. Sun, et al. “Metamorphic Relation Generation: State of the Art and Research Directions.” *ACM TOSEM*. DOI 10.1145/3708521. Preprint arXiv:2406.05397.
- [23] A. Dev, M. Sloan, B. Kavner, S. Kong, and M. Sandler. “Judge Reliability Harness: Stress Testing the Reliability of LLM Judges.” arXiv:2603.05399. — *Corrected 2026-08-15. This paper does not establish the duplicate-item practice we previously cited it for; it perturbs formatting, paraphrase, verbosity and labels. We have no verified source for that practice and no longer assert one, which weakens a demotion of our own detector.*
- [24] “Brevity Constraints Reverse Performance Hierarchies in Language Models.” arXiv:2604.00025. — *Abstract read; full text not read. Our claim that the reasoning-flag arm is a brevity-constrained arm under another name is our reading and is stated as such.*
- [25] Token-limit incompatibility across OpenAI-compatible servers: `ggml-org/llama.cpp` issue #8634 (max_tokens not respected on the non-chat endpoint); `vllm-project/vllm` issue #11976 (request for a server-side cap). — *Search results; the issues have not been read in full. Cited to demote our own mechanism to a known integration wart, which is the conservative direction.*

9.11 Datasets

MalwareBazaar ([abuse.ch](https://bazaar.abuse.ch)), <https://bazaar.abuse.ch> — the dated, family-labelled Windows PE samples analysed end to end. Samples are handled only in a scanner-excluded directory and are never committed.

MABEL — Malware Analysis Benchmark for AI/ML (features only, v2.10) — mined offline into the family-fingerprint catalogue and the ATT&CK case corpus. No binaries downloaded.

Ultimate-RAT-Collection — RAT builder and payload binaries, used for the family-fingerprint catalogue and the leakage-free retrieval evaluation’s held-out split.

DikeDataset — raw Windows PE binaries, downloaded and assessed but not catalogued: its labels are coarse

malice scores with no family or technique annotation.

MITRE ATT&CK (Enterprise), <https://attack.mitre.org> — the family-to-technique uses relationships are the per-sample ground truth, derived from `mitre-attack/attack-stix-data` under the ATT&CK Terms of Use.

A Reproducibility

A.1 The model, exactly

Table 20: The local model and inference engine, pinned by digest and commit. Without both, the sampler finding in this appendix cannot be reproduced.

model	Qwen/Qwen3.6-35B-A3B, quantised IQ3_K_R4 by Unsloth
GGUF sha256	d0de70ef...c4ea — computed locally and matching the HuggingFace etag
HF revision	cfd350fd...1f0d
retrieved	2026-05-11
imatrix calibration dataset	named in §2.0
architecture	hybrid recurrent (SSM keys + full_attention_interval=4), confirmed from the GGUF metadata

Naming the imatrix dataset matters: most papers reporting a quantisation level cannot say which calibration produced it, and two IQ3 quants of the same model are not the same artifact.

A.2 The engine, and an honest note about how we know

Engine: `ik_llama.cpp`, commit **eb570eb96689c235933b813693ca28ab9d3d26de** (“*MTP: Avoid per step SSM copy (#1778)*”).

The binary could not identify itself. `llama-server --version` prints `version: 0 (unknown)`, because the build tree was never a git checkout. The commit was recovered from a depth-1 clone vendored elsewhere in the project and then *proved* to describe the build: identical 837-file source lists, and with line endings normalised **exactly one file differs — the generated common/build-info.cpp**.

We report the reconstruction rather than presenting the identifier as though it had been recorded. The transferable lesson is the one we learned the hard way: **build provenance must be captured at build time**. A retracted first version of this row concluded the commit was unrecoverable; it was recovered only because a vendored copy happened to exist.

A.2.1 Serving command

```
llama-server -m Qwen3.6-35B-A3B-IQ3_K_R4.gguf \  
-c 131072 -t 16 -fa on -ctk q8_0 -ctv q8_0 -ngl 999 \  
-ot "blk\.[[1-3][0-9]]\.\.ffn_(up|gate|down)_exps=CPU" \  
--context-shift on --jinja --host 0.0.0.0 --port 8080
```

Two caveats a reader needs. The context is served at **131,072 — half the model’s native 262,144**. And the launch command documented in the repository at one point specified `--n-cpu-moe 36` while the service actually ran 30 blocks on CPU; the command above is what ran.

A.3 Hardware, and the ceiling it imposes

Single host: RTX 5060, 31 GiB RAM, 16 threads. Three measured properties that shape what can be reproduced here:

- **llama-server grows with cumulative requests and does not plateau** — 10.4 GB fresh, 14.8 GB after a single full analyst pass. A long paired run must restart it between arms; at temperature 0 this is measurement-neutral.
- **Serving at 64k instead of 131k does not lower the peak** (14.6 vs 14.8 GB) — the baseline is CPU-resident expert weights, and the growth follows the context a pass actually consumes.
- **Generation rate varies from 162 to 20 tokens/s across requests.** On this hybrid recurrent architecture, llama.cpp writes and erases a **63 MiB recurrent-state checkpoint every few hundred tokens** at ~39k tokens of context. Long-context calls are therefore not slow in proportion — they fall off a cliff.

The Ghidra container is capped at **6 GB** (`mem_limit`). Its JVM `-Xmx4g` bounds the heap only; the database is memory-mapped and measured RSS reached 5.15 GB against that nominal 4 GB cap. Two host lock-ups preceded this limit being added.

The working set does not fit alongside an interactive session, and that is a reproducibility fact rather than an operational one. Model server ~16 GB, analysis worker ~8 GB, disassembly container up to 6 GB: on a 31 GiB host this leaves nothing for a desktop. When we ran a paired study without that headroom, the swap file was exhausted, the model server had **2.3 GB of its own address space paged out**, and one arm then exceeded a 594-second budget on a 16,000-character prompt. A model generating from disk-backed pages is a different instrument from one generating from RAM, and the timings it produces are not comparable to the rest of the table.

Two consequences for anyone reproducing this work:

- **Give the run the machine.** Long paired studies here are run against an otherwise-idle host. This is not a performance recommendation; arms measured under memory pressure are not commensurable with arms that were not.
- **Record the host state per arm, not per study.** Our harnesses capture `MemAvailable`, `SwapFree` and the model server's resident-versus-swapped split at both ends of every arm, and the scoring script excludes arms whose host was degraded. This screen can only be applied forward: for arms already collected without it, the state is gone and the honest label is `unattributable`. That is the label our own halted ablation carries.
- **Start the model server in its own cgroup, not the harness's.** This one cost us three interrupted runs before we read the kernel log carefully enough:

```
task_memcg=/user.slice/.../snap.code.code-*.scope, task=llama-server
Out of memory: Killed process (llama-server) oom_score_adj:1000
snap.code.code-*.scope: Failed with result 'oom-kill'
```

The harness had launched the server as a child process, so it inherited the cgroup of whatever started the harness — an editor's integrated terminal — and its ~16 GB was accounted to the editor's scope. We had already made the server the kernel's preferred victim, and that was the wrong level: choosing *which process* dies cannot change *whose accounting it dies inside*. Launching it as a transient unit with an explicit `MemoryMax` puts the measurement's memory where it belongs and lets the server hit a limit of its own before the host has to arbitrate.

A.4 Data

Table 21: Every committed artifact the paper’s numbers are derived from, with its size and whether it carries per-sample records.

artifact	what it is	committed
temporal_manifest.json	210 dated samples, 7 cohorts, 27 families, metadata only — no binaries	yes
dynamic_cohort_n100.json	the n=100 dynamic cohort: Windows PE only, stratified by year, seed 20260810 , digest recorded	yes
cape_task_ledger_n100.json	sha256 → CAPE task id for all 100 submissions	yes
sink_hint_frequency.json	per-sample hint measurement, n=100 attempted	yes
cape_baseline.json	per-sample CAPE-only scores	yes
consensus_ablation.json, frontier_probe*.json, ...	per-sample arms for every ablation, one file per arm and configuration	yes
parameter_size_series.json	the size series with each arm’s measured reasoning share and why it was included or excluded	yes
judge_contribution_{uncapped,capped}.json	the judge study in both output-cap conditions, per call, with the branch each failed call took	yes
fallback_bundle_content_{uncapped,capped}.json	per-call technique sets on the fallback path against the cascade set each would have received	yes
outbound_parameter_probe.json	which parameters the local server acts on, measured	yes
CAPE reports	~7 MB each, ~660 MB total	no — see §5

Ground truth resolves a MalwareBazaar family signature to an in-repo MITRE uses fixture through an explicit alias map plus a CamelCase-split fallback. It is **uncurated at technique level**, which Threats to Validity treats as a construct-validity limit rather than a footnote.

Per-sample outputs are stored for every study in this paper. This is a policy adopted after it cost us a result: an earlier study’s per-sample outputs were not retained, and when a defect was later found that could have affected it, the question could not be asked. That study is withdrawn.

A.5 The sandbox, and three things that will block a reader

CAPE 2.5 at a private address, reachable only from one network.

1. **One analysis machine** (win10, x64), no Linux VM. Only Windows PE can be analysed dynamically whatever the code’s stated scope, which is why the cohort is Windows-only. An ELF submitted there joins the 10,348 tasks that are never scheduled.
2. **Reports are retained for days, not indefinitely.** Sampled across the full task-id range, **18 of 18** older tasks return "Reports directory does not exist"; a report from four days earlier survived. The local archive of fetched reports is therefore **the only copy**, and “submit the same samples” is not a reproduction recipe on its own.
3. **The instance rate-limits, and its throttle response is byte-identical to its auth refusal** — {"error": true, "message": ""} in both cases. A harness that reads that string as a capability verdict will mis-classify a throttle as a missing permission; we did, briefly. Re-call a tool that worked minutes earlier to tell them apart.

Timing, for planning: a new Windows submission is scheduled in **~1 second** and completes in **~5.5 minutes**; the 10,348-task backlog is never scheduled and does not queue ahead of new work. An n=100 cohort is roughly 9 hours of sandbox time.

A.6 The comparison endpoints, and one parameter that decides everything

Three endpoints, and a reader reproducing any of them needs the configuration axis before the model names, because it is worth more F1 than any of them.

Table 22: The inference endpoints each arm ran against, with the configuration measured rather than the configuration requested.

arm	endpoint	model	reasoning	n
local baseline	ik_llama.cpp, this host	Qwen3.6-35B-A3B (IQ3_K_R4)	off	25
frontier	OpenRouter	nvidia/nemotron-3-super-120b-a12b:free	on — not controllable	25 ×2
same weights, hosted	DashScope International	qwen3.6-35b-a3b	off	25
same weights, hosted	DashScope International	qwen3.6-35b-a3b	on	25
larger, hosted	DashScope International	qwen3.6-plus	off / on	25 each

enable_thinking is honoured on DashScope and ignored on OpenRouter. Not rejected — accepted, recorded in the result file, and not acted on: the re-run requesting it returned a **56.2%** reasoning share against **56.5%** without it. On DashScope the same request produces **0.0%**. This is the single most important line in this appendix for anyone comparing arms, because the flag is worth 0.34–0.45 F1 on the two models where it can be set, and an arm can therefore be labelled matched while running the opposite configuration. `eval_parameter_size_series.py` selects arms on the **measured** reasoning share for exactly this reason and refuses to correlate when fewer than three configuration-matched arms span three parameter counts — which, on these endpoints, is always.

The local server's output cap must be sent twice. ChatOpenAI(max_tokens=N) reaches the wire as max_completion_tokens, which ik_llama.cpp does not read; the cap must also travel in extra_body as max_tokens and n_predict. Measured on this host: 48 requested, **2805 generated** with the renamed field alone, **48** with both. A reader who omits this will find the verdict model decoding to the context limit, which is what happened here for as long as the cap had existed. `probe_outbound_parameters.py` re-checks this and the other outbound parameters against a running server; it is the regression witness, and it is cheap enough to run before any measurement campaign.

Note also that this server **truncates silently**: at the cap it returns finish_reason: "stop", never "length", and carries no stopped_limit field. Telemetry that detects truncation by finish reason will report zero however often the cap binds; compare the generated-token count against the cap that was requested instead.

A.6.1 The original frontier arm

OpenRouter, nvidia/nemotron-3-super-120b-a12b:free, temperature 0, 2,400-token output cap.

The free tier allows 50 requests per day (X-RateLimit-Limit: 50, limit_source: openrouter_free_tier_daily), resetting daily, with a 20-requests-per-minute cap that applies whether or not credit has been purchased. The **first** attempt at this arm exhausted the daily quota mid-flight — 9 calls landing and 16 returning HTTP 429 — and the n=9 estimate it produced was not merely imprecise but pointed the wrong way (§2). The arm reported above is the completed **n=25** run, taken from a fresh quota: 25 of 25 parsed, 0 refusals, \$0 charged against the \$25 ceiling.

The quota is stated here because it governs what a cohort-scale frontier arm costs in wall-clock rather than in dollars: one call per sample over a 97-sample cohort is two days at 50 requests per day, and a full-pipeline arm — roughly 20 model requests per analysis — is not reachable on the free tier at all. The spend meter is enforced as a precondition on every call rather than reconciled afterwards, and projections price output at the full cap because a degenerate decode produces exactly that.

A.7 Reproducing each result

All harnesses are committed under `tests/evaluation/` and are checkpointed and resumable — several of the runs in this paper were interrupted by the memory guard and resumed without loss.

Table 23: Which harness produced which result, and the command that reruns it.

result	harness	services needed
consensus ablation (§1)	<code>eval_consensus_ablation.py</code>	llama-server
frontier and hosted arms (§2)	<code>eval_frontier_probe.py --arm</code> <code><name> [--no-thinking]</code>	network only
parameter-size series (§2)	<code>eval_parameter_size_</code> <code>series.py</code>	none — reads the arm files
CAPE baseline (§0)	<code>eval_cape_baseline.py</code>	none — reads the local report archive
sink-hint frequency (§4)	<code>eval_sink_hint_frequency.py</code>	Ghidra
sink-hint ablation (§7)	<code>eval_sink_hint_ablation.py</code>	llama-server + Ghidra
opcode-hash attribution (§3)	<code>eval_function_hash_</code> <code>attribution.py</code>	Ghidra + Qdrant
Layer-0 verdict arms (§5)	<code>eval_layer0_verdict.py</code> , checked by <code>verify_b3_mechanism.py</code>	llama-server; the check needs none
cascade weight sensitivity (§5)	<code>eval_weight_sensitivity_</code> <code>six.py</code>	none — reads the report archive
judge contribution (§5)	<code>eval_judge_contribution.py</code>	llama-server
fallback bundle content (§5)	<code>eval_fallback_bundle_</code> <code>content.py --checkpoint</code> <code><run></code>	none — reads the run above
outbound parameter check every figure	<code>probe_outbound_parameters.py</code> <code>make_paper_figures.py</code>	llama-server none — reads the JSON the above emit

Two of these exist only because a result could not be believed, and both are worth running before the studies they check rather than after. `verify_b3_mechanism.py` recomputes each ablation arm’s cascade set from its seeded fixture and compares it against what the arm recorded — it is what established that a published finding of ours had measured a post-processing step rather than a model. `probe_outbound_parameters.py` asks whether the server acts on each parameter we send it, by behaviour rather than by reading a response field, because a server that ignores a parameter does not report having done so.

The judge-contribution study is run **twice, in two conditions**, and the pair is kept: once with the output cap not reaching the server and once with it binding. The comparison between them is what isolates what the model’s unparsable output contributes to the analyst’s bundle, and neither run alone shows it. Preserved as `judge_contribution_uncapped.*` and `judge_contribution_capped.*`.

Figures are generated from the same retained records as the text. The script recomputes intervals with the seeded bootstrap rather than reading them out of a summary, so a figure and a sentence cannot drift apart without the

script failing to reproduce one of them. This caught a live error: an ROC computed by naive descending sort returned AUC 0.458 against the 0.550 in our text, because 186 of 210 claims are tied at confidence 1.0 and tie handling decides the entire estimate. The text was right and the first draft of the figure was wrong — which is the ordinary direction for this check to fire, and the reason for wiring it this way.

Three harness properties are deliberate and worth copying. Every sweep **restarts the server it depends on between samples**, because a read timeout leaves the Ghidra JVM mid-analysis and every subsequent load is refused — a whole window of samples was lost to this before it was understood. Every sweep reports **how many distinct outputs the N inputs produced**, because that one line is the cheapest detector for the stale-state defects described in Threats to Validity. And every sweep records the **host's memory state at both ends of each unit of work**, so that a failed unit can afterwards be attributed to the pipeline or excluded as an artefact of the machine — a question that cannot be reopened later, as §3 above describes.